

Lecture-#0

Natural Language Processing

A8706

Introduction

Syllabus Details



Learning from Experience, Sharing with Passion

Introduction: What is Natural Language Processing (NLP), Origins of NLP, The Challenges of NLP, Phases of NLP, Language and Grammar. Finding the Structure of Words and Documents: Words and Their Components, Issues and Challenges, Morphological Models. Finding the Structure of Documents: Introduction, Sentence Boundary Detection, Topic Boundary Detection, Methods, Complexity of the Approaches, Performances of the Approaches, Features, Processing Stages.

Syntax: Parsing Natural Language, A Data-Driven Approach to Syntax, Stop words, Correcting Words, Stemming, Lemmatization, Parts of Speech (POS) Tagging, Representation of Syntactic Structure, Parsing Algorithms, Models for Ambiguity Resolution in Parsing.

Syllabus Details



Learning from Experience, Sharing with Passion

Semantic Parsing: Introduction, Semantic Interpretation: Structural Ambiguity, Entity and Event Resolution, System Paradigms, Word Sense, Predicate-Argument Structure, Meaning Representation.

Language modelling: Introduction, n-Gram Models, Language Model Evaluation, Parameter Estimation, Types of Language Models: Class-Based Language Models, MaxEnt Language Models, Neural Network Language Models Language-Specific Modeling Problems, Multilingual and Crosslingual Language Modeling.

Applications: Question Answering: History, Architectures, Question Analysis, Search and Candidate Extraction, Automatic Summarization: Approaches to Summarization, Spoken Dialog Systems: Speech Recognition and Understanding, Speech Generation, Dialog Manager, Voice User Interface, Information Retrieval: Document Preprocessing, Monolingual Information Retrieval

Syllabus Details Lab

1.
 - a. Write a program to Tokenize Text to word using NLTK.
 - b. Write a program to Tokenize Text to Sentence using NLTK.
2.
 - a. Write a program to remove numbers, punctuations, and whitespaces in a file.
 - b. Write a program to Count Word Frequency in a file.
3. Write a program to Tokenize and tag the following sentence using Morphological Analysis in NLP.
4.
 - a. Write a program to get Synonyms from WordNet.
 - b. Write a program to get Antonyms from WordNet.
5.
 - a. Write a program to show the difference in the results of Stemming and Lemmatization.
 - b. Write a program to Lemmatizing Words Using WordNet.
6.
 - a. Write a program to print all stop words in NLP.
 - b. Write a program to remove all stop words from a given text.

Syllabus Details Lab



Learning from Experience, Sharing with Passion

7. Write a Python program to apply Collocation extraction word combinations in the text. Collocation examples are “break the rules,” “free time,” “draw a conclusion,” “keeps in mind,” “get ready,” and so on.
8. Write a Python program to extract Relationship that allows obtaining structured information from unstructured sources such as raw text. Strictly stated, it is identifying relations (e.g., acquisition, spouse, employment) among named entities (e.g., people, organizations, locations). For example, from the sentence “Mark and Emily married yesterday,” we can extract the information that Mark is Emily’s husband.
9. Write a program to print POS and parse tree of a given Text.
10. Write a program to print bigram and Trigram of a given Text.
11. Implement a case study of NLP application.

Reference Books

Text Books:

- Daniel M. Bikel Imed Zitouni., Multilingual Natural Language Processing Applications: From Theory to Practice, IBM Press, 2013.
- Tanveer Siddiqui, U.S. Tiwary., Natural Language Processing and Information Re- trieval, Oxford University, 2008.

Reference Books:

- Daniel Jurafsky and James H Martin., Speech and Language Processing: An introduction to Natural Language Processing, Computational Linguistics and Speech Recognition,2nd Edition, Prentice Hall, 2008.
- James Allen., Natural Language Understanding,2nd Edition, Cummings publishing company,1995.

Courses Outcomes

- ❖ A8706.1 Identify the structure of words and documents for text preprocessing.
- ❖ A8706.2 Choose an approach to parse the given text document.
- ❖ A8706.3 Make use of semantic parsing to capture real meaning of text.
- ❖ A8706.4 Select a language model to predict the probability of a sequence of words.
- ❖ A8706.5 Examine the various applications of NLP.

Introduction to Natural Language Processing

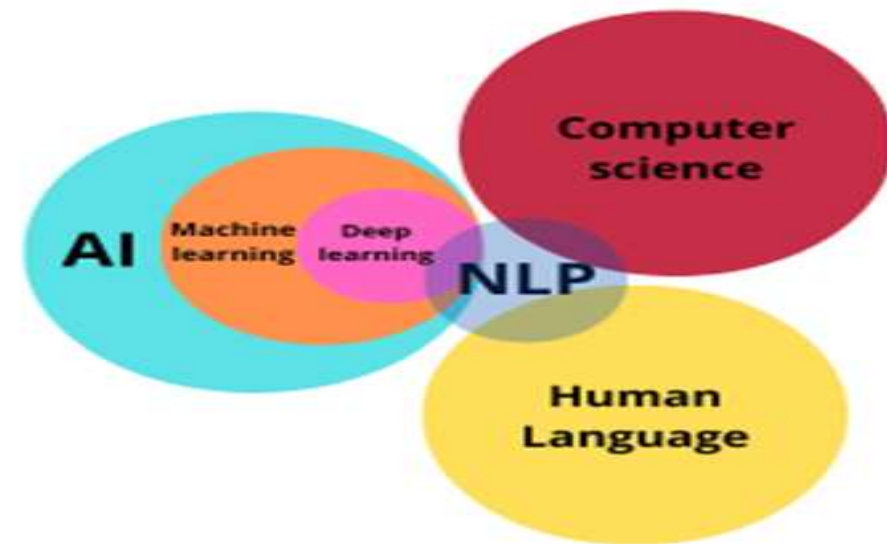


Learning from Experience, Sharing with Passion

Hours Per Week			Hours Per Semester			Credits	Assessment Marks		
L	T	P	L	T	P	C	CIE	SEE	Total
3	0	0	45	0	0	3	40	60	100

Introduction to Natural Language Processing

❖ **Natural language processing (NLP)** refers to the branch of computer science—and more specifically, the branch of artificial intelligence — concerned with giving computers the ability to understand text and spoken words in much the same way human beings can.



Introduction to Natural Language Processing

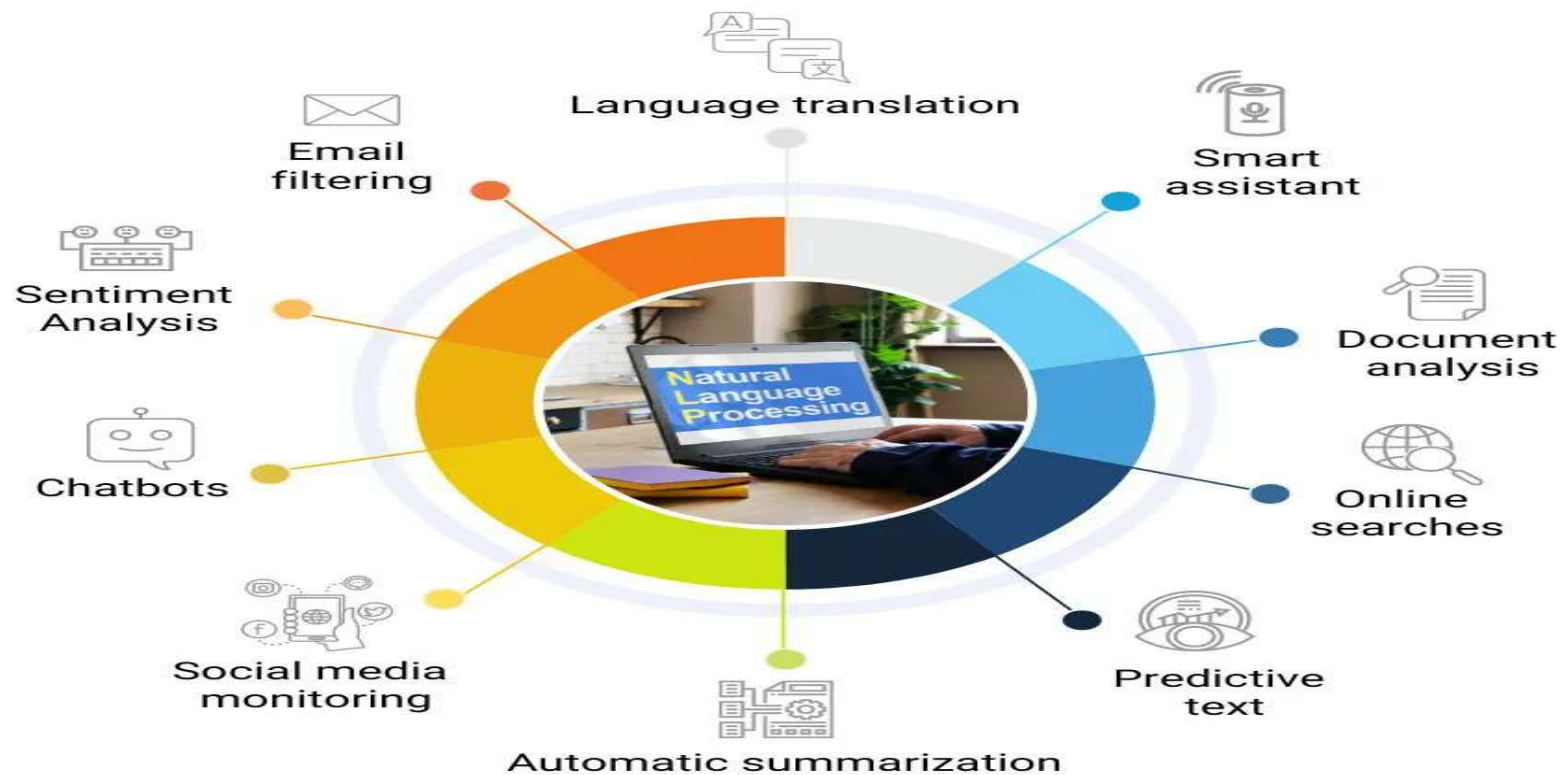


Learning from Experience, Sharing with Passion

- ❖ **NLP** combines computational linguistics—rule-based modeling of human language—with statistical, machine learning, and deep learning models.
- ❖ Together, these technologies enable computers to process human language in the form of text or voice data and to ‘**understand**’ its full meaning, complete with the speaker or writer’s intent and sentiment.
- ❖ **NLP** drives computer programs that translate text from one language to another, respond to spoken commands, and summarize large volumes of text rapidly—even in real time.

Applications of Natural Language Processing

Applications of Natural Language Processing



Applications of Natural Language Processing

1. Sentiment Analysis

- ❖ Sentiment Analysis is also known as **opinion mining**.
- ❖ It is used on the web to analyse the attitude, behaviour, and emotional state of the sender.
- ❖ This application is assigning the values to the text (positive, negative, or neutral), identify the mood of the context (happy, sad, angry, etc.)



Applications of Text Analytics

2 . Question Answering

Question Answering focuses on building systems that automatically answer the questions asked by humans in a natural language



3. Machine Translation

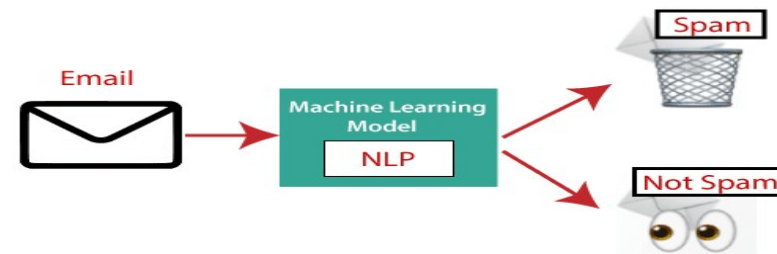
Machine translation is used to translate text or speech from one natural language to another natural language.

Example: Google Translator

Applications of Natural Language Processing

4. Spam Detection

Spam detection is used to detect unwanted e-mails getting to a user's inbox.



5. Spelling and Grammar correction

Microsoft Corporation provides word processor software like MS-word, PowerPoint for the spelling correction.

Example: Grammarly software

Applications of Natural Language Processing



Learning from Experience, Sharing with Passion

6. Speech Recognition

Speech recognition is used for converting spoken words into text. It is used in applications, such as mobile, home automation, video recovery, dictating to Microsoft Word, voice biometrics, voice user interface, and so on.

Example: google Assistant, Siri

7. Chatbot

Implementing the Chatbot is one of the important applications of NLP. It is used by many companies to provide the customer's chat services.

Applications of Natural Language Processing

8. Social media monitoring

It means tracking hashtags, keywords, and mentions relevant to your brand in order to stay informed about your audience and industry.





Lecture-#1

Natural Language Processing

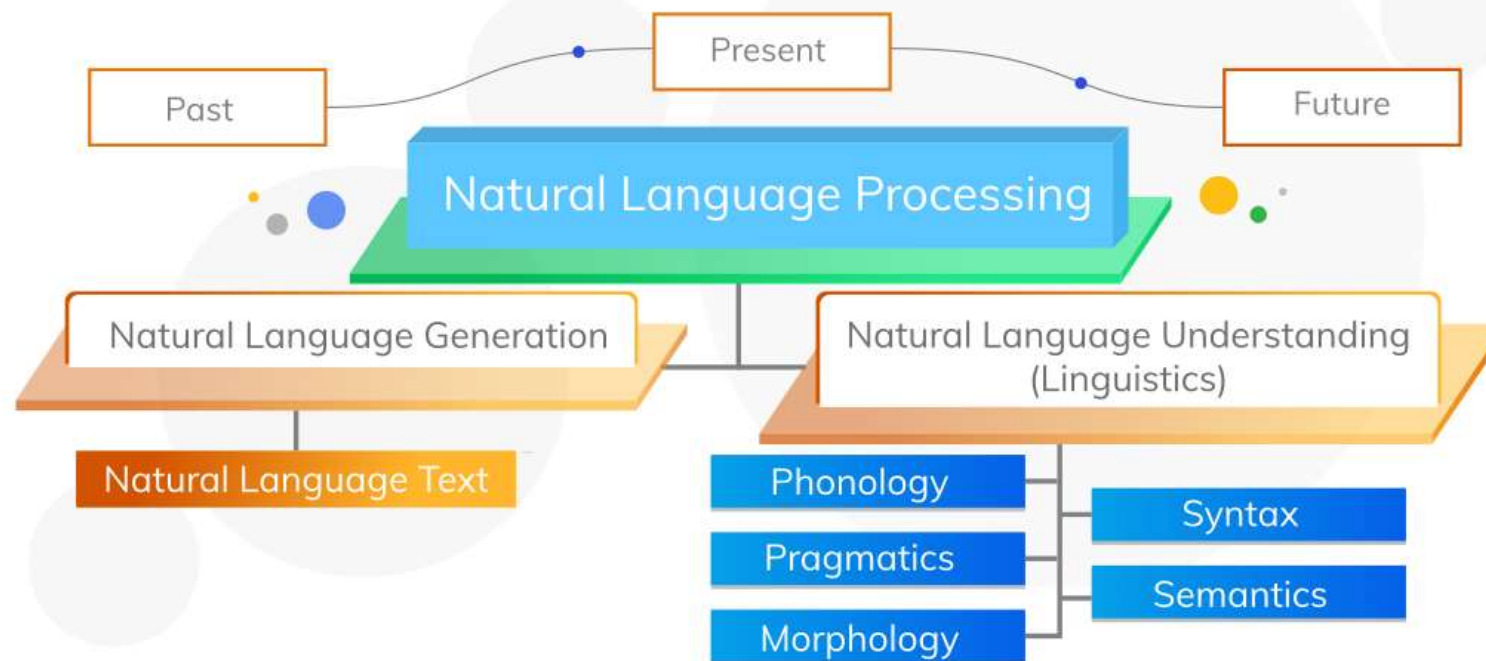
A8706

Introduction

Origins of Natural Language Processing

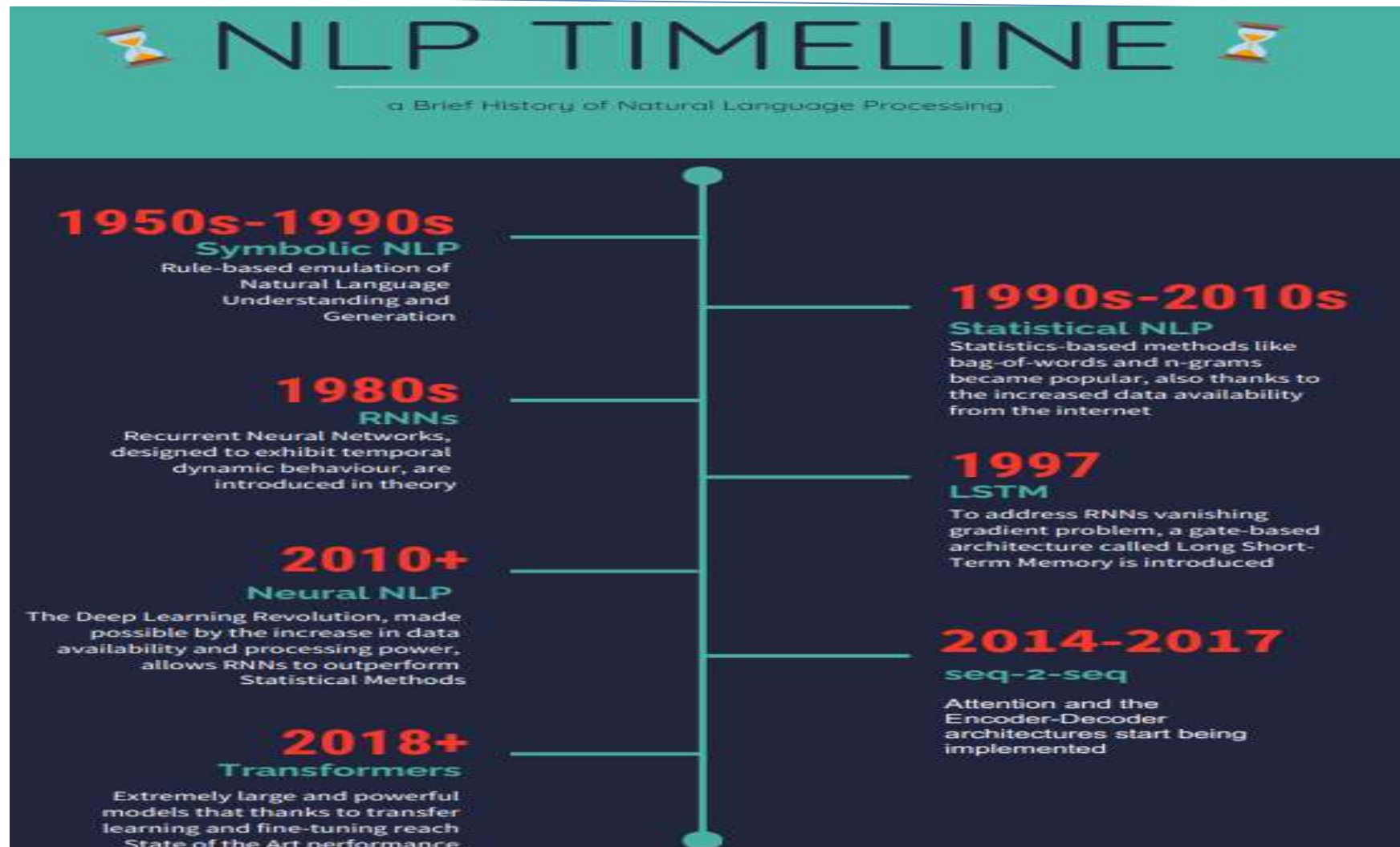


Evolution of NLP



Origins of Natural Language Processing

Alan Turing is the father of Natural language processing. In his **1950** paper **Computing Machinery and Intelligence**, he described a test for an intelligent machine that could understand and respond to natural human conversation.



NLP Terminology

NLP Terminology

Phonology – It is study of organizing sound systematically.

Morphology: The study of the formation and internal structure of words.

Morpheme – It is primitive unit of meaning in a language.

Syntax: The study of the formation and internal structure of sentences.

Semantics: The study of the meaning of sentences.

Pragmatics – It deals with using and understanding sentences in different situations and how the interpretation of the sentence is affected.

Discourse – It deals with how the immediately preceding sentence can affect the interpretation of the next sentence.

World Knowledge – It includes the general knowledge about the world.

Phases of NLP

1. Lexical Analysis and Morphological

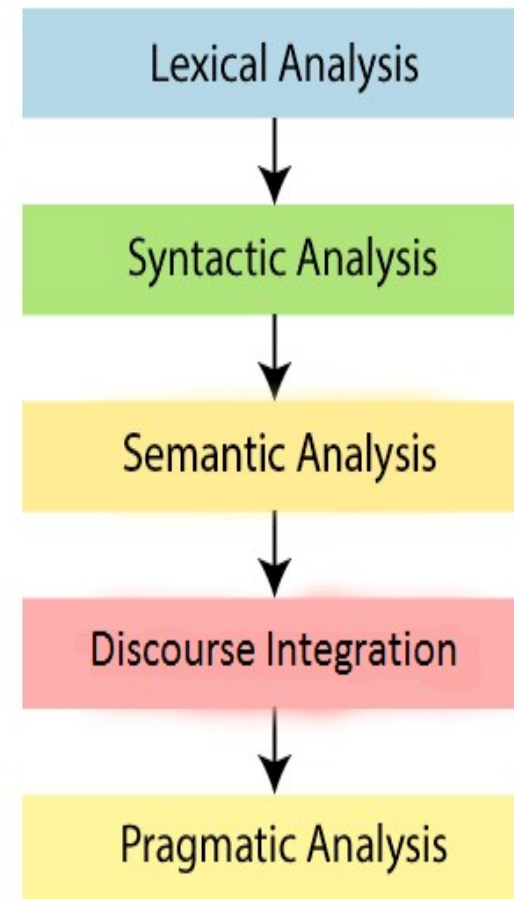
- The first phase of NLP is the Lexical Analysis. This phase scans the source code as a stream of characters and converts it into meaningful lexemes.
- It divides the whole text into paragraphs, sentences, and words.

2. Syntactic Analysis (Parsing)

- Syntactic Analysis is used to check grammar, word arrangements, and shows the relationship among the words.

Example: Agra goes to the Poonam

- In the real world, Agra goes to the Poonam, does not make any sense, so this sentence is rejected by the Syntactic analyzer.



Phases of NLP

3. Semantic Analysis

• Semantic analysis is concerned with the meaning representation. It mainly focuses on the literal meaning of words, phrases, and sentences.

4. Discourse Integration

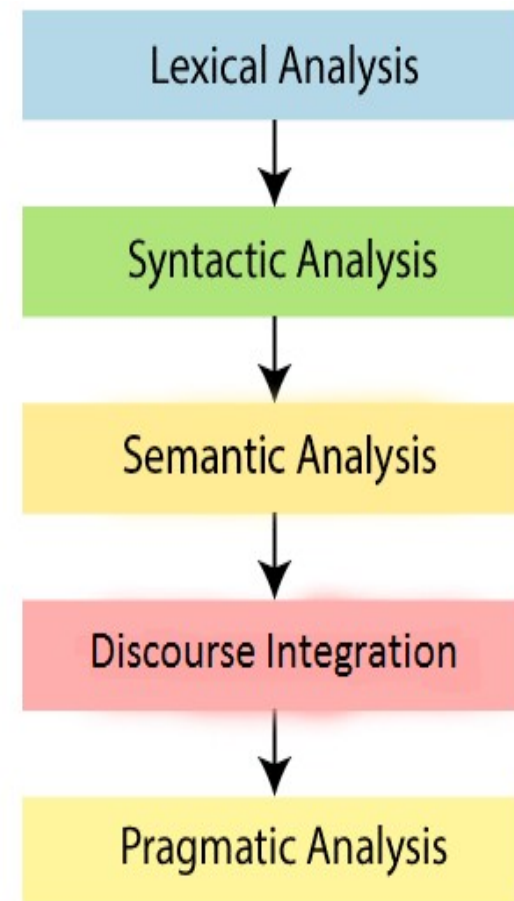
• Discourse Integration depends upon the sentences that precedes it and also invokes the meaning of the sentences that follow it.

• **For example**, the word "that" in the sentence "He wanted that" depends upon the prior discourse context.

5. Pragmatic Analysis

• Pragmatic is the fifth and last phase of NLP. It helps you to discover the intended effect by applying a set of rules that characterize cooperative dialogues.

For Example: "Open the door" is interpreted as a request instead of an order.



The Challenges of NLP

❖ NLP is a powerful tool with huge benefits, but there are still a number of Natural Language Processing limitations and problems:

Contextual words and phrases and homonyms

Synonyms

Ambiguity

Errors in text or speech

Colloquialisms and slang

The Challenges of NLP

❖ Contextual words and phrases and homonyms

1. The same words and phrases can have different meanings according to the context of a sentence and many words – especially in English – have the exact same pronunciation but totally different meanings.

2. Homonyms – two or more words that are pronounced the same but have different definitions – can be problematic for question answering and speech-to-text applications because they aren't written in text form. Usage of *their* and *there*, for example, is even a common problem for humans.

For example:

*I **ran** to the store because we **ran** out of milk.*

*Can I **run** something past you real quick?*

*The house is looking really **run** down.*

These are easy for humans to understand because we read the context of the sentence and we understand all of the different definitions. And, while NLP language models may have learned all of the definitions, differentiating between them in context can present problems.

The Challenges of NLP



Learning from Experience, Sharing with Passion

❖ Synonyms

Synonyms can lead to issues similar to contextual understanding because we use many different words to express the same idea.

Furthermore, some of these words may convey exactly the same meaning, while some may be levels of complexity (small, little, tiny, minute) and different people use synonyms to denote slightly different meanings within their personal vocabulary.

So, for building NLP systems, it's important to include all of a word's possible meanings and all possible synonyms.

Text analysis models may still occasionally make mistakes, but the more relevant training data they receive, the better they will be able to understand synonyms.

The Challenges of NLP

❖ Ambiguity

Ambiguity in NLP refers to sentences and phrases that potentially have two or more possible interpretations.

Lexical ambiguity: a word that could be used as a verb, noun, or adjective. **Example** Light, Fast

Semantic ambiguity: the interpretation of a sentence in context. For example: *I saw the boy on the beach with my binoculars.*

This could mean that I saw a boy through my binoculars or the boy had my binoculars with him

Syntactic ambiguity: In the sentence above, this is what creates the confusion of meaning. The phrase *with my binoculars* could modify the verb, “saw,” or the noun, “boy.”

Even for humans this sentence alone is difficult to interpret without the context of surrounding text.

POS (part of speech) tagging is one NLP solution that can help solve the problem, somewhat.

The Challenges of NLP



Learning from Experience, Sharing with Passion

❖ Colloquialisms and slang

Informal phrases, expressions, idioms, and culture-specific lingo present a number of problems for NLP – especially for models intended for broad use.

Because as formal language, colloquialisms may have no “dictionary definition” at all, and these expressions may even have different meanings in different geographic areas.

Furthermore, cultural slang is constantly morphing and expanding, so new words pop up every day.

This is where training and regularly updating custom models can be helpful, although it oftentimes requires quite a lot of data.

Language and Grammar

Difference between Natural language and Computer Language

Natural Language	Computer Language
Natural language has a very large vocabulary.	Computer language has a very limited vocabulary.
Natural language is easily understood by humans.	Computer language is easily understood by the machines.
Natural language is ambiguous in nature.	Computer language is unambiguous.

❖ NLP is usually used for chatbots, virtual assistants, and modern spam detection.

❖ But NLP isn't perfect, although there are over 7000 languages spoken around the globe, most NLP processes only use seven languages: English, Chinese, Urdu, Farsi, Arabic, French, and Spanish.

Language and Grammar

❖ Grammar in NLP is a set of rules for constructing sentences in a language used to understand and analyze the structure of sentences in text data.

Mathematically, a grammar G can be written as a 4-tuple (N, T, S, P) where,

N or V_N = set of non-terminal symbols, or variables.

T or Σ = set of terminal symbols.

S = Start symbol where $S \in N$

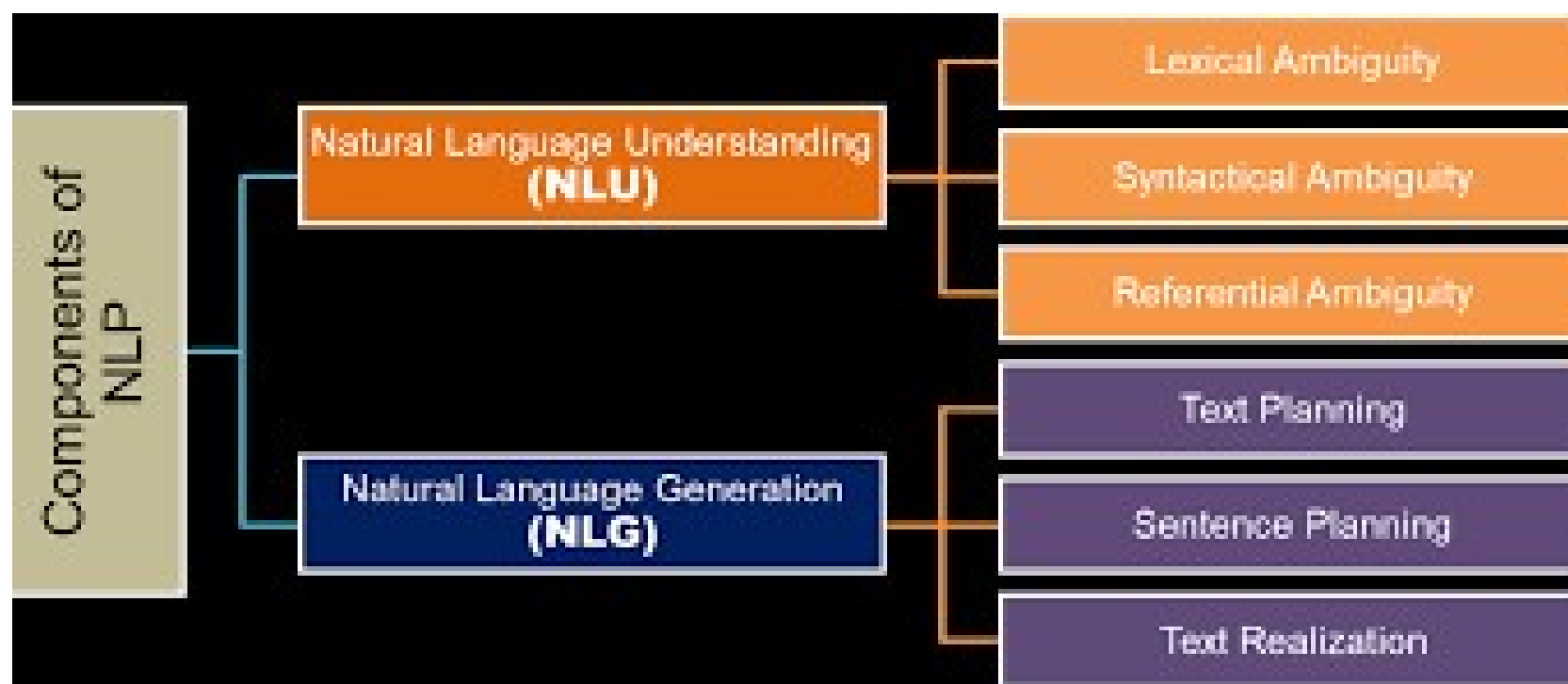
P = Production rules for Terminals as well as Non-terminals.

It has the form $\alpha \rightarrow \beta$, where α and β are strings on $V_N \cup \Sigma$ and at least one symbol of α belongs to V_N



Components of NLP

There are the following two components of NLP –



Components of NLP

Natural language understanding (NLU)

- ❖ NLU enables machines to understand and interpret human language by extracting metadata from content. It performs the following tasks:
- ❖ Helps analyze different aspects of language.
- ❖ Helps map the input in natural language into valid representations.
- ❖ NLU is more difficult than NLG tasks owing to referential, lexical, and syntactic ambiguity.
- ❖ **Lexical ambiguity:** This means that one word holds several meanings. For example, "The man is looking for the match." The sentence is ambiguous as 'match' could mean different things such as a partner or a competition.
- ❖ **Syntactic ambiguity:** This refers to a sequence of words with more than one meaning. For example, "The fish is ready to eat." The ambiguity here is whether the fish is ready to eat its food or whether the fish is ready for someone else to eat. This ambiguity can be resolved with the help of the part-of-speech tagging technique.
- ❖ **Referential ambiguity:** This involves a word or a phrase that could refer to two or more properties. For example, Tom met Jerry and John. They went to the movies. Here, the pronoun 'they' causes ambiguity as it isn't clear who it refers to.

Components of NLP

Natural language generation (NLG)

❖ NLG is a method of creating meaningful phrases and sentences (natural language) from data. It comprises three stages: text planning, sentence planning, and text realization.

❖ **Text planning:** Retrieving applicable content.

❖ **Sentence planning:** Forming meaningful phrases and setting the sentence tone.

❖ **Text realization:** Mapping sentence plans to sentence structures.

❖ [Chatbots](#), machine translation tools, analytics platforms, voice assistants, sentiment analysis platforms, and AI-powered transcription tools are some applications of NLG.

Components of NLP

NLU	NLG
NLU is the process of reading and interpreting language.	NLG is the process of writing or generating language.
It produces non-linguistic outputs from natural language inputs.	It produces constructing natural language outputs from non-linguistic inputs.



Lecture-#2

Natural Language Processing

A8706

Introduction

Text Tokenization

Corpus:

A Corpus is defined as a collection of text documents.

For Example,

A data set containing news is a corpus or
The tweets containing Twitter data are a corpus.

Text Tokenization

Tokens:

Tokens can be words, phrases, or Engrams, and Engrams are defined as the group of n words together.

For example, consider the sentence given below:

```
Sentence: I like my iphone
```

For the above sentence, the different engrams are as follows:

```
Uni-grams( $n=1$ ) are: I, like, my, iphone
```

```
Di-grams( $n=2$ ) are: I like, like my, my iphone
```

```
Tri-grams( $n=3$ ) are: I like my, like my iphone
```

Corpus > Documents > Paragraphs > Sentences > Tokens

Text Tokenization

ngrams:

```
!pip install nltk
```

```
from nltk import ngrams
```

```
sentence = 'Hello everyone. Welcome to class. You are studying Web and Text Analytics article'
```

```
n = 2
```

```
x = ngrams(sentence.split(), n)
```

```
for grams in x:
```

```
    print (grams)
```

O/P:

```
('Hello', 'everyone.') ('everyone.', 'Welcome') ('Welcome', 'to') ('to', 'class.') ('class.', 'You') ('You', 'are') ('are', 'studying') ('studying', 'Web') ('Web', 'and') ('and', 'Text') ('Text', 'Analytics') ('Analytics', 'article')
```

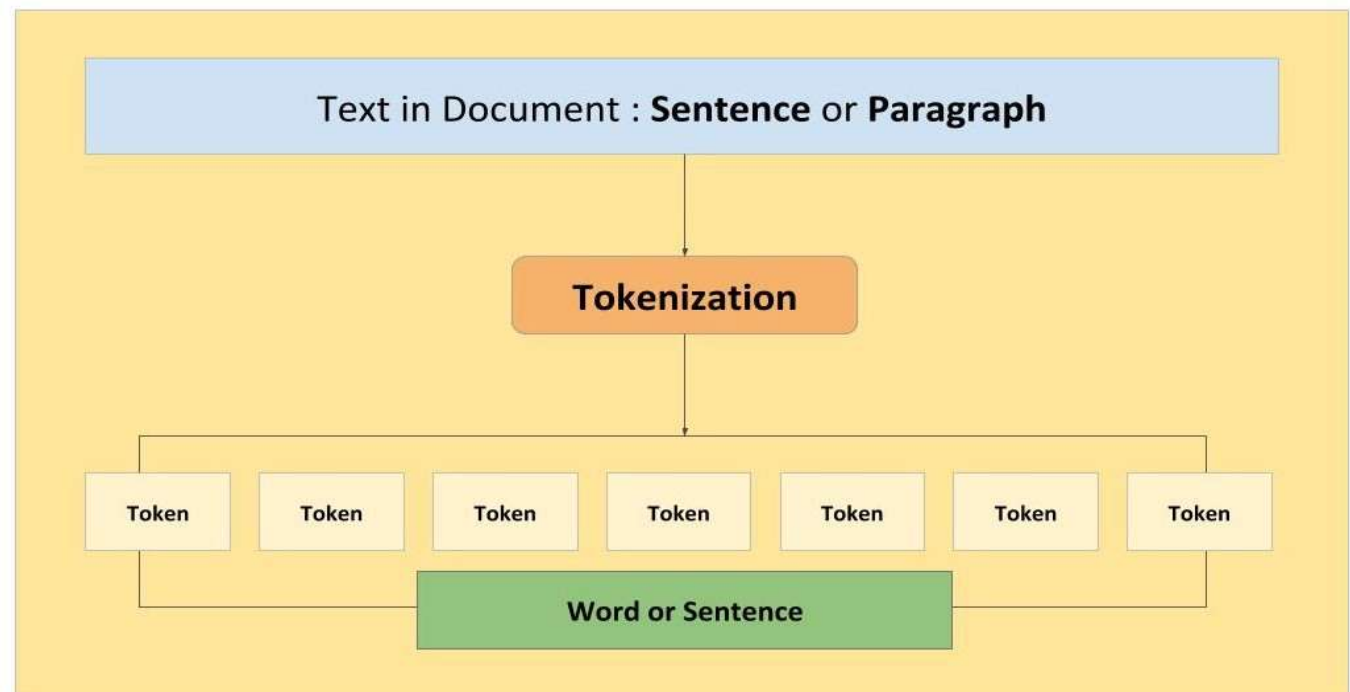
Text Tokenization

Tokenization:

It is the process of converting a text into tokens.

Types:

- ❖ Sentence Tokenization
- ❖ Word Tokenization



Sentence Tokenization

Sentence tokenization is the process of splitting a text corpus into sentences that act as the first level of tokens which the corpus is comprised of. This is also known as *sentence segmentation*, because we try to segment the text into meaningful sentences. Any text corpus is a body of text where each paragraph comprises several sentences.

Sentences are separated by a full stop, so the process of sentence tokenization finds all the full stops in a piece of text to split the data into sentences.

```
!pip install nltk  
import nltk
```

Sentence Tokenization

- ❖ **PUNKT** is an unsupervised trainable model, which means it can be trained on unlabeled data (Data that has not been tagged with information identifying its characteristics, properties, or categories is referred to as unlabeled data.)
- ❖ It contains a variety of libraries for various purposes like text classification, parsing, stemming, tokenizing, etc.
- ❖ **punkt** is the required package for tokenization. Hence you may download it using nltk download manager or download it programmatically using
`nltk.download('punkt').`

Sentence Tokenization

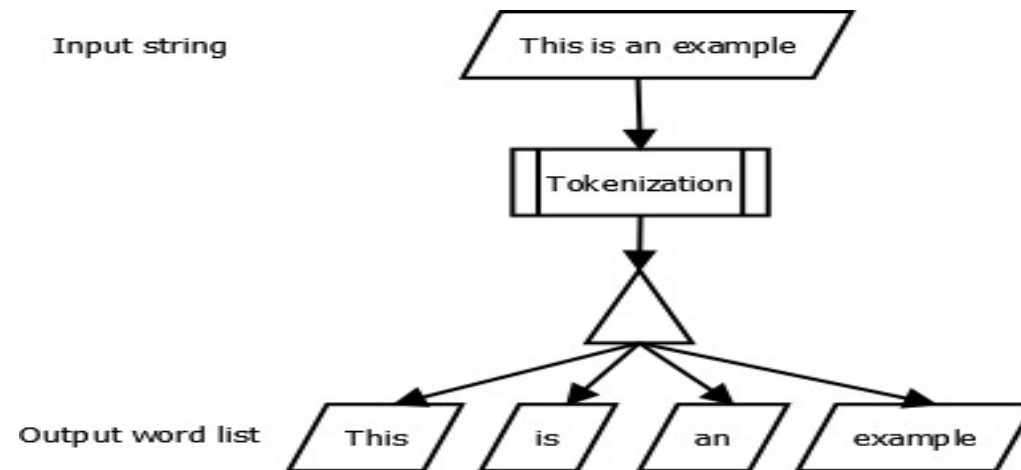
```
!pip install nltk
import nltk
nltk.download('punkt')
from nltk.tokenize import sent_tokenize
text = "Hello everyone. Welcome to class. You are studying Web and Text An
alytics article"
sent_tokenize(text)
```

OUTPUT:

```
['Hello everyone.', 'Welcome to class.', 'You are studying Web and Text Analytics
article']
```

Word Tokenization

- ❖ Word tokenization is the process of splitting a large sample of text into words.
- ❖ This is a requirement in natural language processing tasks where each word needs to be captured and subjected to further analysis like classifying and counting them for a particular sentiment etc.



Word Tokenization

```
from nltk.tokenize import word_tokenize
```

```
text = "Hello everyone. Welcome to Web and Text Analytic class"  
word_tokenize(text)
```

O/P:

```
['Hello', 'everyone', '.', 'Welcome', 'to', 'Web', 'and', 'Text', 'Analytic', 'class']
```

Word Tokenization

Regular expression (RE): It is a set of characters or a pattern that is used to find substrings in a given string.

Example:

Language

$L = \{\epsilon, 00, 0000, 000000, \dots\}$

RE

$R = (00)^*$

Language

$\{\text{google, googoogle, googoogoogle, googooogoogle, ...}\}$

RE

$R = g(oog)^+le$

Word Tokenization

RegexpTokenizer:

```
from nltk.tokenize import RegexpTokenizer
tokenizer2 = RegexpTokenizer(r"[\w]+")
text = "Hello everyone. Welcome to Web and Text Analytic class"
print(tokenizer2.tokenize(text))
```

O/P:

```
['Hello', 'everyone', 'Welcome', 'to', 'Web', 'and', 'Text', 'Analytic',  
'class']
```

Word Tokenization

RegexpTokenizer:

Special Sequences	
<code>\b</code>	Word boundary (zero width)
<code>\d</code>	Any decimal digit (equivalent to <code>[0-9]</code>)
<code>\D</code>	Any non-digit character (equivalent to <code>[^0-9]</code>)
<code>\s</code>	Any whitespace character (equivalent to <code>[\t\n\r\f\v]</code>)
<code>\S</code>	Any non-whitespace character (equivalent to <code>[^ \t\n\r\f\v]</code>)
<code>\w</code>	Any alphanumeric character (equivalent to <code>[a-zA-Z0-9_]</code>)
<code>\W</code>	Any non-alphanumeric character (equivalent to <code>[^a-zA-Z0-9_]</code>)

Word Tokenization

Punctuation:

```
# import string library function  
import string
```

```
# Storing the sets of punctuation in variable result  
result = string.punctuation
```

```
# Printing the punctuation values  
print(result)
```

O/P:

```
!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

Word Tokenization

WordPunctTokenizer:

WordPunctTokenizer is used for separating the punctuation from the words.

```
from nltk.tokenize import WordPunctTokenizer  
tokenizer1 = WordPunctTokenizer()  
tokenizer1.tokenize(text)
```

O/P:

```
['Hello', 'everyone', '.', 'Welcome', 'to', 'class', '.', 'You', 'are', 'studying',  
'Web', 'and', 'Text', 'Analytics', 'article']
```

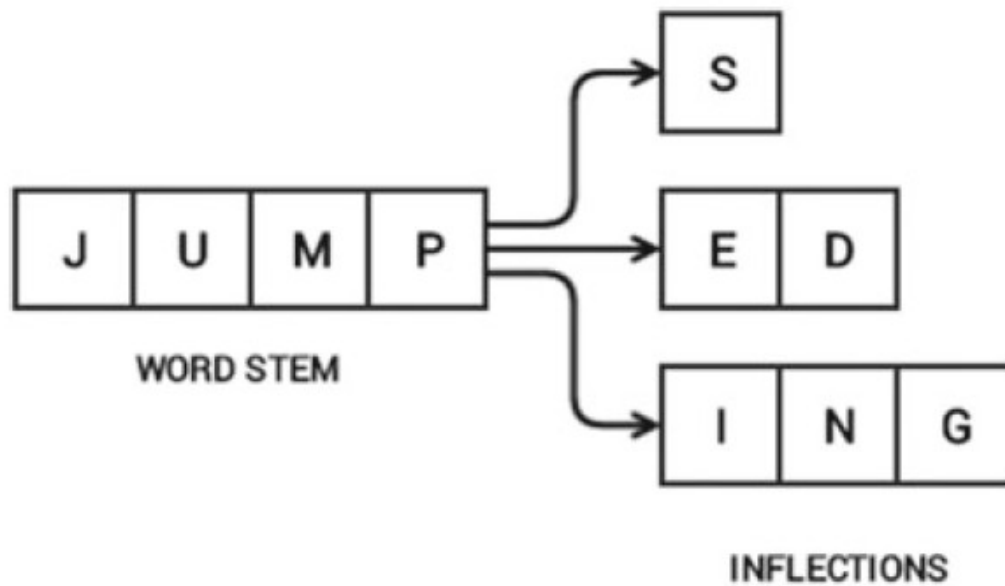
Stemming

Stemming:

- ❖ **Stemming** is the process of producing morphological variants of a root/base word.
- ❖ Stemming programs are commonly referred to as stemming algorithms or stemmers.
- ❖ A stemming algorithm reduces the words “chocolates”, “chocolatey”, and “choco” to the root word, “chocolate” and “retrieval”, “retrieved”, “retrieves” reduce to the stem “retrieve”.
- ❖ **Errors in Stemming:** There are mainly two errors in stemming –
Overstemming and ***Understemming***.
- ❖ Overstemming occurs when two words are stemmed from the same root that are of different stems.
- ❖ Under-stemming occurs when two words are stemmed from the same root that is not of different stems.

Stemming

Stemming:



Stemming

Stemming:

Example: *university* and *universe*

Overstemming: *univers*

data” and “datum

Understemming: *dat* and *datu*

Applications of stemming are:

- ❖ Stemming is used in information retrieval systems like search engines.
- ❖ It is used to determine domain vocabularies in domain analysis.
- ❖ Stemming is desirable as it may reduce redundancy as most of the time the word stem and their inflected/derived words mean the same.

Types of Stemmers in NLTK

Porter's Stemmer

- ❖ It is based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes.

Example: EED -> EE means “if the word has at least one vowel and consonant plus EED ending, change the ending to EE” as ‘agreed’ becomes ‘agree’.

- **Advantage:** It produces the best output as compared to other stemmers and it has less error rate.
- **Limitation:** Morphological variants produced are not always real words.

Stemming

```
from nltk.stem import PorterStemmer

# Create a Porter Stemmer instance
porter_stemmer = PorterStemmer()

# Example words for stemming
words = ["running", "jumps", "happily", "running", "happily"]

# Apply stemming to each word
stemmed_words = [porter_stemmer.stem(word) for word in words]

# Print the results
print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

Types of Stemmers in NLTK

Lovins Stemmer

- ❖ removes the longest suffix from a word then the word is recorded to convert this stem into valid words.

Example: sitting -> sitt -> sit

- **Advantage:** It is fast and handles irregular plurals like 'teeth' and 'tooth' etc.
- **Limitation:** It is time consuming and frequently fails to form words from stem.

Stemming

```
pip install stemming  
from stemming.lovins import stem
```

```
word = "jumping"  
stemmed_word = stem(word)
```

```
print(stemmed_word)
```

Types of Stemmers in NLTK

Dawsons Stemmer

- ❖ It is an extension of Lovins stemmer in which suffixes are stored in the reversed order indexed by their length and last letter. Example: sitting -> sitt -> sit
- **Advantage:** It is fast in execution and covers more suffices.
- **Limitation:** It is very complex to implement.

Types of Stemmers in NLTK

Krovetz Stemmer

- 1) Convert the plural form of a word to its singular form.
- 2) Convert the past tense of a word to its present tense and remove the suffix 'ing'.

Example: 'children' -> 'child'

Advantage: It is light in nature and can be used as pre-stemmer for other stemmers.

Limitation: It is inefficient in case of large documents.

Types of Stemmers in NLTK

Xerox Stemmer

Capable of processing extensive datasets and generating valid words, it has a tendency to over-stem, primarily due to its reliance on lexicons, making it language-dependent. This constraint implies that its effectiveness is limited to specific languages.

Example:

'children' -> 'child'

'understood' -> 'understand'

'whom' -> 'who'

'best' -> 'good'

Types of Stemmers in NLTK

N-Gram Stemmer

The algorithm, aptly named n-grams (typically $n=2$ or 3), involves breaking words into segments of length n and then applying statistical analysis to identify patterns. An n-gram is a set of n consecutive characters extracted from a word in which similar words will have a high proportion of n-grams in common.

Example: 'INTRODUCTIONS' for $n=2$ becomes : *I, IN, NT, TR, RO, OD, DU, UC, CT, TI, IO, ON, NS, S*

- **Advantage:** It is based on string comparisons and it is language dependent.
- **Limitation:** It requires space to create and index the n-grams and it is not time efficient.

Types of Stemmers in NLTK

Snowball Stemmer

The Snowball stemmer is way more aggressive than Porter Stemmer and is also referred to as Porter2 Stemmer. Because of the improvements added when compared to the Porter Stemmer, the Snowball stemmer is having greater computational speed.

- * multi-lingual as it can handle non-English words.

Types of Stemmers in NLTK

Lancaster Stemmer

The Lancaster stemmers are more aggressive and dynamic compared to the other two stemmers. The stemmer is really faster, but the algorithm is really confusing when dealing with small words. But they are not as efficient as Snowball Stemmers. The Lancaster stemmers save the rules externally and basically uses an iterative algorithm.

Types of Stemmers in NLTK

Regex Stemmer

The Regex Stemmer, or Regular Expression Stemmer, is a stemming algorithm that utilizes regular expressions to identify and remove suffixes from words. It allows users to define custom rules for stemming by specifying patterns to match and remove.

Stemming

Stemming:

```
# import these modules
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
ps = PorterStemmer()
# choose some words to be stemmed
words = ["program", "programs",
"programmer", "programming",
"programmers"]
for w in words:
    print(w, " : ", ps.stem(w))
```

O/P:

program : program

programs : program

programmer : programm

programming : program

programmers : programm

Stemming

Stemming:

```
# importing modules
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

ps = PorterStemmer()

sentence = "Programmers program with
programming languages"
words = word_tokenize(sentence)
for w in words:
    print(w, " : ", ps.stem(w))
```

O/P:

Programmers : programm
program : program
with : with
programming : program
languages : languag

Lemmatization

Lemmatization:

- ❖ Lemmatization is the process of grouping together the different inflected forms of a word so they can be analyzed as a single item.
- ❖ Lemmatization is similar to stemming but it brings context to the words.
- ❖ So it links words with similar meanings to one word.

❖ Example:

- ❖ -> rocks : rock
- ❖ -> corpora : corpus
- ❖ -> better : good

Applications of lemmatization are:

- ❖ Used in comprehensive retrieval systems like search engines.
- ❖ Used in compact indexing

Lemmatization

Lemmatization:

likes



like

better



good

worse



bad

Word	Stemming	Lemmatization
information	inform	information
informative	inform	informative
computers	comput	computer
feet	feet	foot

Lemmatization

Techniques:

Rule Based Lemmatization

❖ It involves the application of predefined rules to derive the base or root form of a word. Unlike machine learning-based approaches, which learn from data, rule-based lemmatization relies on linguistic rules and patterns.

•**Rule:** For regular verbs ending in “-ed,” remove the “-ed” suffix.

Example:

Word: “walked”

Rule Application: Remove “-ed”

Result: “walk

Lemmatization

Techniques:

Dictionary-Based Lemmatization

❖ It relies on predefined dictionaries or lookup tables to map words to their corresponding base forms or lemmas. Each word is matched against the dictionary entries to find its lemma. This method is effective for languages with well-defined rules.

Example:

'running' -> 'run'

'better' -> 'good'

'went' -> 'go'

Lemmatization

Techniques:

Machine Learning -Based Lemmatization

❖ Unlike rule-based or dictionary-based approaches, machine learning models, such as neural networks or statistical models, are trained on large text datasets to generalize patterns in language.

❖ Example:

'running' -> 'run'

'better' -> 'good'

'went' -> 'go'

Lemmatization

Lemmatization:

```
# import these modules  
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()
```

```
print("rocks :", lemmatizer.lemmatize("rocks"))  
print("corpora :", lemmatizer.lemmatize("corpora"))
```

```
# a denotes adjective in "pos"
```

```
print("better :", lemmatizer.lemmatize("better", pos="a"))
```

O/P:

rocks : rock

corpora : corpus

better : good

Lemmatization

Advantages:

Improves text analysis accuracy

Reduces data size

Better search results

Disadvantages:

Time-consuming

Not suitable for real-time applications

May lead to ambiguity

O/P:

rocks : rock

corpora : corpus

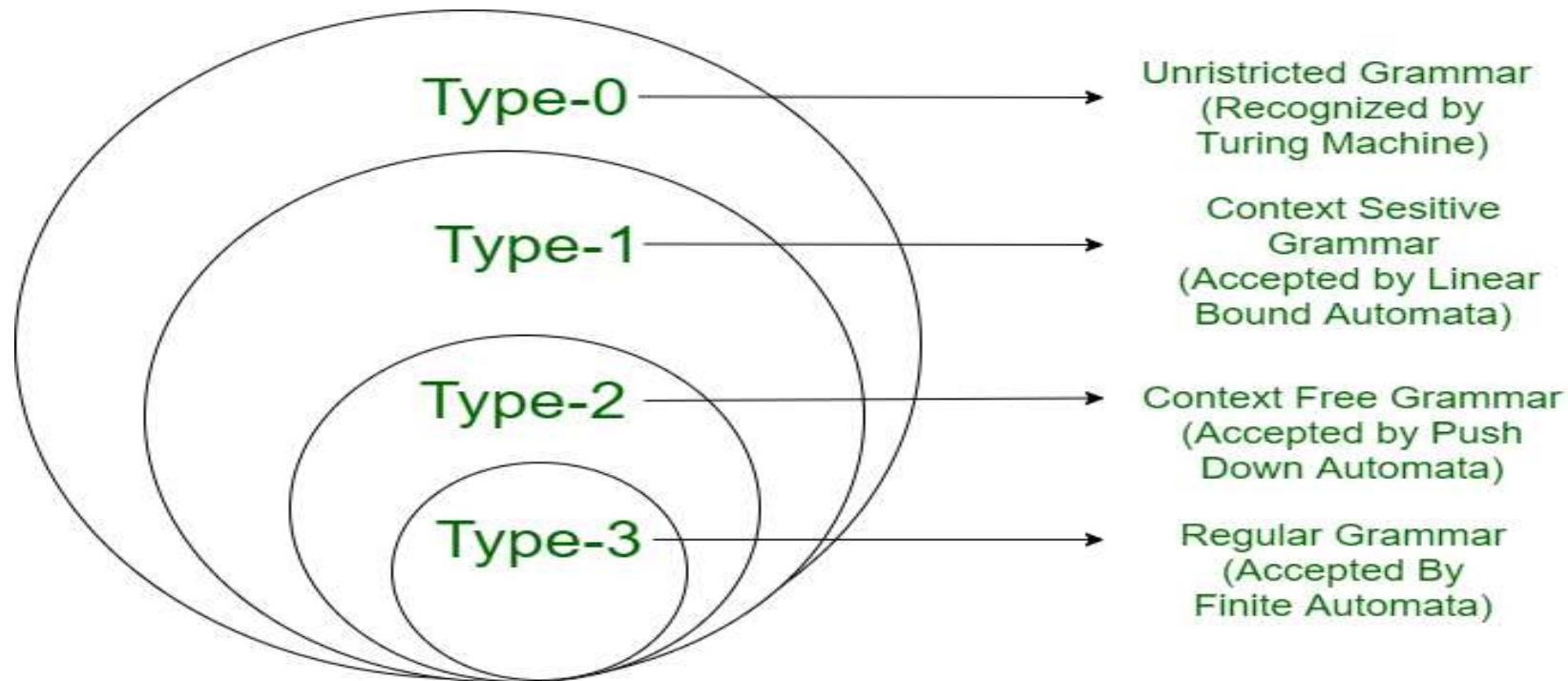
better : good

STEMMING vs LEMMATIZATION

	<u>Stemming</u>	<u>Lemmatization</u>
1	Stemming is faster because it chops words without knowing the context of the word in given sentences.	Lemmatization is slower as compared to stemming but it knows the context of the word before proceeding.
2	It is a rule-based approach.	It is a dictionary-based approach.
3	Accuracy is less.	Accuracy is more as compared to Stemming.
4	When we convert any word into root-form then stemming may create a word with no meaning.	Lemmatization always gives the dictionary meaning while converting words into root-form.
5	It is used when the meaning of the word is not important. Example: Spam Detection	It should be used when it is important to retain the meaning of the word. Example: Question Answer
6	For Example: "Studies" => "Studi"	For Example: "Studies" => "Study"

Grammar

❖ rules of a language governing the sounds, words, sentences, and other elements, as well as their combination and interpretation.



Context Free Grammar

Context free grammar is a formal grammar which is used to generate all possible strings in a given formal language.

Context free grammar G can be defined by four tuples as:

$G = (V, T, P, S)$

Where

V describes a finite set of non-terminal symbols or variables

T describes a finite set of terminal symbols.

P describes a set of production rules

S is the start symbol.

Context Free Grammar

Context free grammar is a formal grammar which is used to generate all possible strings in a given formal language.

Context free grammar G can be defined by four tuples as:

$G = (V, T, P, S)$

Where

V describes a finite set of non-terminal symbols or variables

T describes a finite set of terminal symbols.

P describes a set of production rules

S is the start symbol.

Example

The grammar $(\{A\}, \{a, b, c\}, P, A)$, $P : A \rightarrow aA, A \rightarrow abc$.

The grammar $(\{S\}, \{a, b\}, P, S)$, $P : S \rightarrow aSa, S \rightarrow bSb, S \rightarrow \epsilon$

The grammar $(\{S, F\}, \{0, 1\}, P, S)$, $P : S \rightarrow 00S \mid 11F, F \rightarrow 00F \mid \epsilon$

Context Free Grammar

Generation of Derivation Tree

A derivation tree or parse tree is an ordered rooted tree that graphically represents the semantic information a string derived from a context-free grammar.

Representation Technique

Root vertex – Must be labeled by the start symbol.

Vertex – Labeled by a non-terminal symbol.

Leaves – Labeled by a terminal symbol or ϵ .

Leftmost derivation – A leftmost derivation is obtained by applying production to the leftmost variable in each step.

Rightmost derivation – A rightmost derivation is obtained by applying production to the rightmost variable in each step.

Context Free Grammar

Generation of Derivation Tree or parse tree

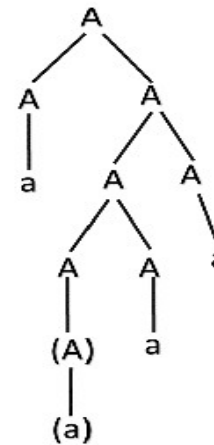
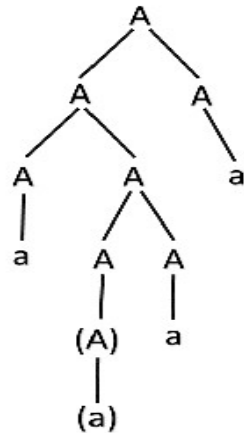
$A \rightarrow AA$

$A \rightarrow (A)$

$A \rightarrow a$

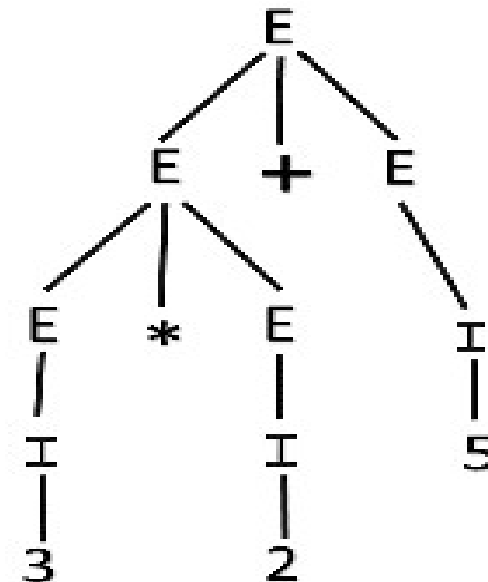
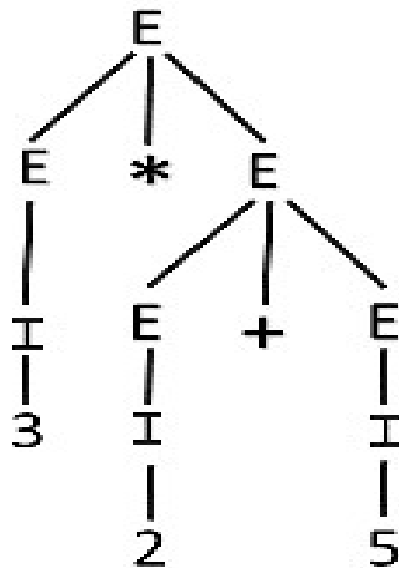
Solution:

For the string "a(a)aa" the above grammar can generate two parse trees:



Context Free Grammar

A CFG is ambiguous if one or more terminal strings have multiple leftmost derivations from the start symbol.



Parts of Speech (POS) Tagging

❖ part-of-speech tagging (POS tagging or PoS tagging or POST), also called grammatical tagging or word-category disambiguation.

❖ Input: I am going.

❖ Output:

with stop word

[('I', 'PRP'), ('am', 'VBP'), ('going', 'VBG'), ('.', '.')]]

Without stop word

[('I', 'PRP'), ('going', 'VBG'), ('.', '.')]]

Where

PRP- Pronoun Personal

VBP-Verb non-3rd personal singular present

VBG-Verb Gerund/ Present Participle

Parts of Speech (POS) Tagging

Sl No.	TAG	DESCRIPTION	EXAMPLE(S)
1	CC	Coordinating Conjunction	<i>and, or</i>
2	CD	Cardinal Number	<i>five, one, 2</i>
3	DT	Determiner	<i>a, the</i>
4	EX	Existential <i>there</i>	<i>there were two cars</i>
5	FW	Foreign Word	<i>d'hoevre, mais</i>
6	IN	Preposition/ Subordinating Conjunction	<i>of, in, on, that</i>
7	JJ	Adjective	<i>quick, lazy</i>
8	JJR	Adjective, comparative	<i>quicker, lazier</i>

Parts of Speech (POS) Tagging

9	JJS	Adjective, superlative	<i>quickest, laziest</i>
10	LS	List item marker	<i>2)</i>
11	MD	Verb, modal	<i>could, should</i>
12	NN	Noun, singular or mass	<i>fox, dog</i>
13	NNS	Noun, plural	<i>foxes, dogs</i>
14	NNP	Noun, proper singular	<i>John, Alice</i>
15	NNPS	Noun, proper plural	<i>Vikings, Indians, Germans</i>
16	PDT	Predeterminer	<i>both the cats</i>
17	POS	Possessive ending	<i>boss's</i>
18	PRP	Pronoun, personal	<i>me, you</i>

Parts of Speech (POS) Tagging

19	PRP\$	Pronoun, possessive	<i>our, my, your</i>
20	RB	Adverb	<i>naturally, extremely, hardly</i>
21	RBR	Adverb, comparative	<i>better</i>
22	RBS	Adverb, superlative	<i>best</i>
23	RP	Adverb, particle	<i>about, up</i>
24	SYM	Symbol	<i>%, \$</i>
25	TO	Infinitival to	<i>how to, what to do</i>
26	UH	Interjection	<i>oh, gosh, wow</i>

Parts of Speech (POS) Tagging

Sl No.	TAG	DESCRIPTION	EXAMPLE(S)
27	VB	Verb, base form	<i>run, give</i>
28	VBD	Verb, past tense	<i>ran, gave</i>
29	VBG	Verb, gerund/ present participle	<i>running, giving</i>
30	VBN	Verb, past participle	<i>given</i>
31	VBP	Verb, non-3rd person singular present	<i>I think, I take</i>
32	VBZ	Verb, 3rd person singular present	<i>he thinks, he takes</i>
33	WDT	Wh-determiner	<i>which, whatever</i>
34	WP	Wh-pronoun, personal	<i>who, what</i>
35	WP\$	Wh-pronoun, possessive	<i>whose</i>

Parts of Speech (POS) Tagging

36	WRB	Wh-adverb	<i>where, when</i>
37	NP	Noun Phrase	<i>the brown fox</i>
38	PP	Prepositional Phrase	<i>in between, over the dog</i>
39	VP	Verb Phrase	<i>was jumping</i>
40	ADJP	Adjective Phrase	<i>warm and snug</i>
41	ADVP	Adverb Phrase	<i>also</i>
42	SBAR	Subordinating Conjunction	<i>whether or not</i>
43	PRT	Particle	<i>up</i>
44	INTJ	Interjection	<i>hello</i>
45	PNP	Prepositional Noun Phrase	<i>over the dog, as of today</i>
46	-SBJ	Sentence Subject	<i>the fox jumped over the dog</i>
47	-OBJ	Sentence Object	<i>the fox jumped over the dog</i>

Parts of Speech (POS) Tagging



Learning from Experience, Sharing with Passion

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt = " Everything is all about money."
# sent_tokenize is one of instances of
# PunktSentenceTokenizer from the nltk.tokenize.punkt module
tokenized = sent_tokenize(txt)
for i in tokenized:
    # Word tokenizers is used to find the words
    # and punctuation in a string
    wordsList = nltk.word_tokenize(i)
# removing stop words from wordList
wordsList = [w for w in wordsList if not w in stop_words]
# Using a Tagger. Which is part-of-speech
# tagger or POS-tagger.
tagged = nltk.pos_tag(wordsList)
print(tagged)
```

O/P:

```
[('Everything', 'VBG'),
('money', 'NN'),
('.', '.')]

```

Parts of Speech (POS) Tagging

with stop words

```
import nltk
# download required nltk packages
# required for tokenization
nltk.download('punkt')
# required for parts of speech tagging
nltk.download('averaged_perceptron_tagger')

# input text
sentence = """"Today morning, I am Very Happy.""""

# tokene into words
tokens = nltk.word_tokenize(sentence)

# parts of speech tagging
tagged = nltk.pos_tag(tokens)

# print tagged tokens
print(tagged)
```

O/P:

```
[('Today', 'NN'),
('morning', 'NN'),
(',', ','),
('I', 'PRP'),
('am', 'VBP'),
('Very', 'RB'),
('Happy', 'JJ'), (',', ',')]
```

Parts of Speech (POS) Tagging

```
# Without stop words
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt = "Today morning, I am Very Happy."
# sent_tokenize is one of instances of
# PunktSentenceTokenizer from the nltk.tokenize.punkt module
tokenized = sent_tokenize(txt)
for i in tokenized:
# Word tokenizers is used to find the words
# and punctuation in a string
    wordsList = nltk.word_tokenize(i)
    # removing stop words from wordList
    wordsList = [w for w in wordsList if not w in stop_words]
    # Using a Tagger. Which is part-of-speech
    # tagger or POS-tagger.
    tagged = nltk.pos_tag(wordsList)
    print(tagged)
```

O/P:

```
[('Today', 'NN'),
('morning', 'NN'),
(',', ','), ('I', 'PRP'),
('Very', 'RB'),
('Happy', 'JJ'), (':', ':')]
```




Lecture-#3

Natural Language Processing

A8706

Introduction

Finding the Structure of Words and Documents

Natural Language Processing Pipeline



Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- Human language is a complicated thing.
- We use it to express our thoughts, and through language, we receive information and infer its meaning.
- Trying to understand language all together is not a viable approach.
- Linguists have developed whole disciplines that look at language from different perspectives and at different levels of detail.
- The point of **morphology**, for instance, is to study the variable forms and functions of words,
- The syntax is concerned with the arrangement of words into phrases, clauses, and sentences.
- Word structure constraints due to pronunciation are described by **phonology**,
- The conventions for writing constitute the **orthography** of a language.
- The meaning of a linguistic expression is its semantics, and etymology and lexicology cover especially the evolution of words and explain the semantic, morphological, and other links among them.
- Words are perhaps the most intuitive units of language, yet they are in general tricky to define.
- Knowing how to work with them allows, in particular, the development of **syntactic** and **semantic** abstractions and simplifies other advanced views on language.
- Here, first we explore how to identify words of distinct types in human languages, and how the internal structure of words can be modelled in connection with the grammatical properties and lexical concepts the words should represent.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- The discovery of word structure is **morphological parsing**.
- In many languages, words are delimited in the orthography by whitespace and punctuation.
- But in many other languages, the writing system leaves it up to the reader to tell words apart or determine their exact phonological forms.

Words and Their Components

- Words are defined in most languages as the smallest linguistic units that can form a complete utterance by themselves.
- The minimal parts of words that deliver aspects of meaning to them are called **morphemes**.

Tokens

- Suppose, for a moment, that words in English are delimited only by whitespace and punctuation (the marks, such as full stop, comma, and brackets)
- Example: **Will you read the newspaper? Will you read it? I won't read it.**

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

If we confront our assumption with insights from syntax, we notice two here: words *newspaper* and *won't*.

- Being a compound word, *newspaper* has an interesting **derivational structure**.
- In writing, *newspaper* and the associated concept is distinguished from the isolated *news* and *paper*.
- For reasons of generality, linguists prefer to analyze *won't* as two syntactic words, or tokens, each of which has its independent role and can be reverted to its normalized form.
- The structure of *won't* could be parsed as *will* followed by *not*.
- In English, this kind of tokenization and **normalization** may apply to just a limited set of cases, but in other languages, these phenomena have to be treated in a less trivial manner.
- In Arabic or Hebrew, certain tokens are concatenated in writing with the preceding or the following ones, possibly changing their forms as well.
- The underlying lexical or syntactic units are thereby blurred into one compact string of letters and no longer appear as distinct words.
- Tokens behaving in this way can be found in various languages and are often called **clitics**.
- In the writing systems of Chinese, Japanese, and Thai, whitespace is not used to separate words.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Lexemes

- By the term word, we often denote not just the one **linguistic form** in the given context but also the **concept behind the form** and the **set of alternative forms** that can express it.
- Such **sets** are called **lexemes or lexical items**, and they constitute the **lexicon** of a language.
- Lexemes can be divided by their behaviour into the lexical categories of verbs, nouns, adjectives, conjunctions, particles, or other parts of speech.
- The citation **form of a lexeme**, by which it is commonly identified, is also called its **lemma**.
- When we convert a word into its other forms, such as turning the **singular *mouse*** into the **plural *mice or mouses***, we say we **inflect** the lexeme.
- When we transform a lexeme into another one that is morphologically related, regardless of its lexical category, we say we derive the lexeme: for instance, the nouns ***receiver* and *reception*** are derived from the verb *to receive*.
- Example: **Did you see him? I didn't see him. I didn't see anyone.**
- Example presents the problem of tokenization of ***didn't*** and the investigation of the internal structure of ***anyone***.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- In the paraphrase *I **saw no one***, the lexeme *to see* would be inflected into the form **saw** to reflect its grammatical function of expressing **positive past tense**.
- Likewise, *him* is the oblique case form of *he* or even of a more abstract lexeme representing all personal pronouns.
- In the paraphrase, ***no one*** can be perceived as the minimal word synonymous with *nobody*.
- The difficulty with the definition of what counts as a word need not pose a problem for the syntactic description if we understand ***no one as two closely connected tokens treated as one fixed element***.

Morphemes

- Morphological theories differ on whether and how to associate the properties of word forms with their structural components.
- These components are usually called **segments** or **morphs**.
- The morphs that by themselves represent some aspect of the meaning of a word are called **morphemes** of some function.
- Human languages employ a variety of devices by which morphs and morphemes are combined into word forms.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Morphology

- Morphology is the domain of linguistics that analyses the internal structure of words.
- Morphological analysis – exploring the structure of words
- Words are built up of minimal meaningful elements called **morphemes**: **played** = **play**-**ed**

cats = **cat**-**s**

unfriendly = **un**-**friend**-**ly**

- Two types of morphemes: i Stems: **play**, **cat**, **friend** ii Affixes: **-ed**, **-s**, **un-**, **-ly**

- Two main types of affixes:

i Prefixes precede the stem: **un-**

ii Suffixes follow the stem: **-ed**, **-s**, **un-**, **-ly**

- Stemming = find the stem by stripping off affixes **play** = **play**

replayed = **re**-**play**-**ed**

computerized = **comput**-**er**-**ize**-**d**

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Problems in morphological processing

- **Inflectional morphology:** inflected forms are constructed from base forms and inflectional affixes.
- Inflection relates different forms of the same word

Lemma	Singular	Plural
cat	cat	cats
dog	dog	dogs
knife	knife	knives
sheep	sheep	sheep
mouse	mouse	mice

- **Derivational morphology:** words are constructed from roots (or stems) and derivational affixes:

inter+national = international international+ize = internationalize

internationalize+ation = internationalization

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- The simplest morphological process concatenates morphs one by one, as in dis- agree-ment-s, where agree is a free lexical morpheme and the other elements are bound grammatical morphemes contributing some partial meaning to the whole word.
- In a more complex scheme, morphs can interact with each other, and their forms may become subject to additional phonological and orthographic changes denoted as morphophonemic.
- The alternative forms of a morpheme are termed allomorphs.

Typology

- Morphological typology divides languages into groups by characterizing the prevalent morphological phenomena in those languages.
- It can consider various criteria, and during the history of linguistics, different classifications have been proposed.
- Let us outline the typology that is based on quantitative relations between words, their morphemes, and their features:
- **Isolating**, or **analytic**, languages include no or relatively few words that would comprise more than one morpheme (typical members are Chinese, Vietnamese, and Thai; analytic tendencies are also found in English).

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

• **Synthetic** languages can combine more morphemes in one word and are further divided into agglutinative and fusional languages.

Example:

"Domovinosios" (fictional word in a synthetic language inspired by Slavic languages)

- Dom: root meaning "home"
- -ovi: infix denoting "belonging to"
- -nos: suffix denoting plural possession
- -ios: suffix indicating genitive case
- **"Domovinosios"** might be translated as "of the homes" in English

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- **Agglutinative** languages have morphemes associated with only a single function at a time (as in Korean, Japanese, Finnish, and Tamil, etc.)

- **Example:**

"Evlerinizden"

Ev: root meaning "house"

-ler: suffix for plural, meaning "houses"

-iniz: possessive suffix for "your," meaning "your houses"

-den: ablative case suffix, meaning "from"

In English, **"Evlerinizden"** translates to "from your houses."

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

• **Fusional** languages are defined by their feature-per-morpheme ratio higher than one (as in Arabic, Czech, Latin, Sanskrit, German, etc.).

"Gacchāmi" (गच्छामि)

Gacch: root verb meaning "go"

-ā: suffix indicating the first person singular

-mi: suffix indicating present tense

In this single word, "Gacchāmi" means "I go" or "I am going."

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- **Concatenative** languages linking morphs and morphemes one after another.

"Unhappiness"

Un-: prefix meaning "not" or "opposite of"

Happy: root word meaning "joyful" or "content"

-ness: suffix that turns the adjective "happy" into a noun indicating a state or quality

In this word, "Unhappiness" is formed by concatenating the prefix "Un-", the root "Happy", and the suffix "-ness".

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- **Nonlinear** languages allowing structural components to merge nonsequentially to apply tonal morphemes or change the consonantal or vocalic templates of words.

- ❖ root-and-pattern morphology

- The root **k-t-b** (which generally relates to writing):

- **Kataba** (كَتَبَ): “He wrote”

- Root: **k-t-b**
- Pattern: **a-a-a**

- **Kutiba** (كُتِبَ) “It was written”

- Root: **k-t-b**
- Pattern: **u-i-a**

- **Kitaab** (كِتَاب) : “Book”

- Root: **k-t-b**
- Pattern: **i-aa**

- In these examples, the consonantal root **k-t-b** remains constant, while different vowel patterns (and sometimes additional affixes) are applied to convey different meanings.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Morphological Typology

- **Morphological typology** is a way of classifying the languages of the world that groups languages according to their common [morphological](#) structures.
- The field organizes languages on the basis of how those languages form [words](#) by combining [morphemes](#).
- The morphological typology classifies languages into **two broad classes** of **synthetic languages** and **analytical languages**.
- The **synthetic class** is then further sub classified as either **agglutinative languages** or **fusional languages**.
- [Analytic](#) languages contain very little [inflection](#), instead relying on features like [word order](#) and auxiliary words to convey meaning.
- [Synthetic](#) languages, ones that are not analytic, are divided into two categories: [agglutinative](#) and [fusional](#) languages.
- Agglutinative languages rely primarily on discrete particles([prefixes](#), [suffixes](#), and [infixes](#)) for inflection, ex: inter+national = international, international+ize = internationalize.
 - While fusional languages "fuse" inflectional categories together, often allowing one word ending to contain several categories, such that the original root can be difficult to extract (anybody, newspaper).



Lecture-#4

Natural Language Processing

A8706

Introduction

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Issues and Challenges

- **Irregularity:** word forms are not described by a prototypical linguistic model.
 - **Ambiguity:** word forms be understood in multiple ways out of the context of their discourse.
 - **Productivity:** is the inventory of words in a language finite, or is it unlimited?
-
- **Irregularity:** Irregularity in language refers to situations where word forms do not follow the typical or regular patterns that might be expected based on linguistic rules or models.
 - Instead, irregular words have unique or idiosyncratic forms that don't conform to standard morphological or phonological rules. Here are some key points regarding irregularity in language:
 - Morphological Irregularity:** Irregularity often manifests in the way words are formed. For example, in English, the past tense of regular verbs is typically formed by adding "-ed" (e.g., "walk" becomes "walked"). However, irregular verbs like "go" have unique past tense forms ("went") that do not follow this pattern.
 - Phonological Irregularity:** Irregularity can also occur at the phonological level, affecting the pronunciation of words. For instance, the pronunciation of some irregularly spelled words in English may not align with their spelling (e.g., "colonel" is pronounced as "kernel").

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Ambiguity: Word structure can introduce ambiguity in language. The same word may have multiple meanings depending on its context and the affixes used.

For example, the word "**unhinged**" can mean "**mentally unstable**" or "**detached from its hinges**," illustrating how a single word can have different meanings based on its structure.

Lexical Ambiguity: Lexical ambiguity occurs when a single word has multiple meanings, and the context does not make it clear which meaning is intended. Examples include:

"Bank" can mean a financial institution or the side of a river.

"Bat" can refer to a flying mammal or a piece of sports equipment.

Syntactic Ambiguity: Syntactic ambiguity arises from the structure or arrangement of words within a sentence. Different interpretations of the sentence can result from alternative ways of parsing it. Examples include:

"I saw the man with the telescope." (Did I use the telescope to see the man, or did the man have the telescope?)

"The chicken is ready to eat." (Is the chicken ready to be eaten, or is it prepared to eat something else?)

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Semantic Ambiguity: Semantic ambiguity occurs when a sentence or phrase is vague or imprecise due to multiple possible interpretations of its meaning. Examples include:

"She saw the man with glasses." (Does "with glasses" refer to the man wearing glasses or the glasses themselves?)

"He's a little chicken." (Does "chicken" mean cowardly or a small bird?)

Pragmatic Ambiguity: Pragmatic ambiguity arises from the context or implied meaning of a statement. It can occur when a statement is technically clear but can be interpreted in different ways based on the speaker's intention or the situation. Examples include:

"Could you pass the salt?" (Is this a request for someone to pass the salt, or is it a polite way of saying, "I would like some salt, please"?)

"I am not saying he's a liar." (Is the speaker denying that the person is a liar, or are they implying something indirectly?)

Homonyms: Homonyms are words that have the same spelling or pronunciation but different meanings. This can lead to lexical ambiguity. Examples include:

"Bear" (referring to the animal) and "bear" (meaning to carry or endure).

"Tear" (referring to ripping something) and "tear" (referring to a drop of liquid from the eye).

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Productivity:

- ❖ Not all word formations are equally productive in a language. Some affixes are more versatile and can be applied to a wide range of words, while others have limited applicability.
- ❖ Understanding which affixes are productive and which are not is important for linguistic analysis and language learning.

Original word: **happy**

New word: **unhappy** (not happy)

Original word: **do**

New word: **undo** (reverse the action of doing)

Original word: **likely**

New word: **unlikely** (not likely)

Original word: **fair**

New word: **unfair** (not fair)

Original word: **able**

New word: **unable** (not able)

Original word: **pleasant**

New word: **unpleasant** (not pleasant)

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Morphological Models

- ❖ Morphological models in linguistics are theoretical frameworks or approaches that are used to describe and analyze the structure of words in a language.
- ❖ These models help linguists understand how words are formed, how they inflect, and how their internal components (morphemes) contribute to their meanings and functions.
- ❖ There are several morphological models, each with its own way of categorizing and analyzing linguistic elements. Here are some of the key morphological models:

• Dictionary Lookup

- Dictionary lookup is a practical application of morphological models in the context of language and linguistics.
- While dictionary lookup itself doesn't explicitly employ a specific morphological model, it relies on the underlying principles of morphology to provide users with information about words and their structures.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Here's how dictionary lookup relates to morphological models :

- **Word Structure:** Morphological models describe how words are structured, including the identification of morphemes (the smallest meaningful units of language) and their combinations to form words.
- When you perform a dictionary lookup, the information you receive often includes details about a word's morphological structure. For example, you may learn about the word's root, affixes, and how they contribute to its meaning.
- **Inflection and Derivation:** Morphological models distinguish between inflectional and derivational morphemes.
- Inflectional morphemes convey grammatical information such as tense, number, and case, while derivational morphemes create new words or change word categories (e.g., noun to verb).
- Dictionaries often indicate whether a word has undergone inflectional changes and may provide derivational information about related words or word forms.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Consider the English verb "**to run**":

Inflectional Forms:

Base form: **run**

Third-person singular: **runs**

Past tense: **ran**

Present participle: **running**

Past participle: **run**

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

➤ Word Forms:

Nouns: Typically, nouns have singular and plural forms (e.g., cat vs. cats). Some languages also have other forms, like dual in Arabic.

Verbs: Verbs often have multiple forms to express different tenses, aspects, moods, or voices (e.g., walk vs. walked vs. walking).

Adjectives: Adjectives might change form to express degrees of comparison (e.g., happy, happier, happiest).

➤ **Lexical Ambiguity:** Morphological models help explain the different meanings that a word can have based on its morphological structure.

➤ When you look up a word in a dictionary, you may encounter multiple definitions or senses of the word, and understanding the morphological components can aid in disambiguating these meanings.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Are Dictionary Lookup and Orthography the Same?

No, they are *not the same*, but they are *closely related* in morphological analysis.

Orthography-based Morphological Analysis

- **Orthography** deals with the **written form of words**.
- In morphological analysis, it means **breaking a word into parts (prefix, root, suffix) based on spelling patterns**.
- It **does not require meaning** — it's all about the **surface structure**.

Example:

"unhappily" → "un" + "happi" + "ly"

Even if "happi" is not a valid word, it's just based on **what remains after removing affixes**.

- **Focus:** Word structure by form
Used for: Rule-based stemmers, language teaching, quick heuristics

Dictionary Lookup in Morphology

- Here, you **check each word or morpheme against a known lexicon** (i.e., a dictionary).
- You **only recognize meaningful units** that exist in your dictionary.
- It verifies if the remaining **root or affix is a valid morpheme**.

Example:

"redo" → check if "re" is a known prefix and "do" is a valid root in dictionary.

Focus: Validity by meaning

Used for: Lemmatization, morphological analyzers (like spaCy, NLTK)

Finite-State Morphology

- Finite-State Morphology (FSM) is a computational linguistic model used for describing and generating complex word forms and morphological processes in natural languages. It employs finite-state automata and transducers to represent the morphological rules and processes. FSM is particularly useful for languages with relatively simple and regular morphological systems.
- Here's a simplified example of Finite-State Morphology in English, focusing on the pluralization of nouns:
- By finite-state morphological models, we mean those in which the specifications written by human programmers are directly compiled into finite-state transducers.
- The two most popular tools supporting this approach, XFST (Xerox Finite-State Tool) and Lex Tools.
- Finite-state transducers are computational devices extending the power of finite-state Automata.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Example: Pluralization in English

In English, many nouns form their plurals by adding the suffix "-s" or "-es" to the singular form. Finite-State Morphology can be used to model this process.

Singular Nouns: The FSM model starts with a list of singular nouns, represented as follows:

"cat" -> "cat"

"dog" -> "dog"

"book" -> "book"

"house" -> "house"

Pluralization Rules: The FSM model includes rules for pluralization, which are typically defined as finite-state transducers. In this example, we have two rules:

Rule 1: Add "-s" to the singular form.

Rule 2: Add "-es" to the singular form if it ends with a consonant followed by "y" (e.g., "city" -> "cities").

Plural Nouns: The FSM applies the pluralization rules to the singular nouns to generate the plural forms:

Applying Rule 1 to "cat" -> "cats"

Applying Rule 1 to "dog" -> "dogs"

Applying Rule 1 to "book" -> "books"

Applying Rule 2 to "city" -> "cities"

Applying Rule 2 to "house" -> "houses"

Finding the Structure of Words and Documents

Input	Input Morphological parsed output
Cats	cat +N +PL
Cat	cat +N +SG
Cities	city +N +PL
Geese	goose +N +PL
Goose	goose +N +SG) or (goose +V)
Geese	goose +V +3SG
merging	merge +V +PRES-PART
Caught	(caught +V +PAST-PART) or (catch +V +PAST)

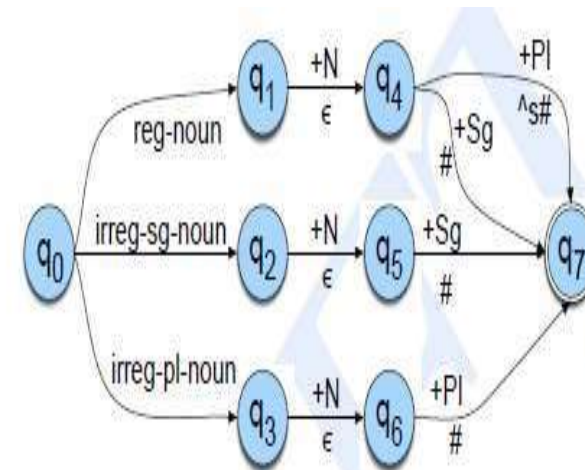


Figure 3.13 A schematic transducer for English nominal number inflection T_{num} . The symbols above each arc represent elements of the morphological parse in the lexical tape; the symbols below each arc represent the surface tape (or the intermediate tape, to be described later), using the morpheme-boundary symbol ϵ and word-boundary marker $\#$. The labels on the arcs leaving q_0 are schematic, and need to be expanded by individual words in the lexicon.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- The FSM model captures the regularity of English pluralization by representing it as a set of finite-state rules and applying these rules to generate plural forms.
- While this example is highly simplified, FSM models can be extended to cover more complex morphological processes in various languages.
- FSM is commonly used in computational linguistics and natural language processing tasks, such as stemming, lemmatization, and inflection generation, where it can efficiently handle large sets of word forms and their associated morphological rules.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Unification-Based Morphology

- Unification-Based Morphology (UBM) is a linguistic model that represents morphological processes using unification, a concept from formal logic and artificial intelligence.
- UBM focuses on the idea that morphological processes can be described as a series of feature unifications, where features of morphemes come together to form words.
- Morphological parsing P thus associates linear forms ϕ with alternatives of structured content ψ , cf.

I have seen two cats walks down the street

$$\mathcal{P} :: \phi \rightarrow \{\psi\}$$

$$\mathcal{P} :: form \rightarrow \{content\} \quad (1.2)$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- ❖ unification can be monotonic (i.e., information-preserving), or it can allow inheritance of default values and their overriding.
- ❖ In either case, information in a model can be efficiently shared and reused by means of inheritance hierarchies defined on the feature structure types.'
- ❖ Unification is the key operation by which feature structures can be merged into a more informative feature structure.
- ❖ Unification of feature structures can also fail, which means that the information in them is mutually incompatible.
- ❖ Morphological models of this kind are typically formulated as logic programs, and unification is used to solve the system of constraints imposed by the model.
- ❖ Advantages of this approach include better abstraction possibilities for developing a morphological grammar as well as elimination of redundant information from it.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Unification:

- ❖ Unification is a process of merging two or more sets of features (attribute-value pairs) to form a single, consistent set. If the features are compatible, unification succeeds; if there is a conflict (e.g., a feature expecting both singular and plural values), unification fails.
- ❖ In morphology, unification is used to combine features from different morphemes or morphological rules to produce a complete word form or to analyze an existing word form.

Feature Structures:

- ❖ Feature structures are the primary data representations in unification-based systems. They consist of attributes (e.g., "number," "tense," "case") and their corresponding values (e.g., "singular," "past," "nominative").
- ❖ Feature structures can be complex, hierarchical, and recursive, allowing for the detailed representation of morphological, syntactic, and semantic information.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Morphological Rules:

- ❖ In unification-based morphology, morphological rules are expressed as constraints on feature structures.
- ❖ These rules specify how morphemes combine and what features must agree (e.g., subject-verb agreement in number and person).
- ❖ For example, a rule might state that a noun's number feature must match the number feature of an affix (e.g., plural suffix "-s" in English).

Handling Agreement and Inflection:

- ❖ Unification is particularly powerful in handling agreement phenomena, where different parts of a sentence must share certain grammatical features. For instance, subject-verb agreement in English requires the subject and verb to agree in number and person.
- ❖ Unification-based morphology can also handle complex inflectional systems, where multiple features interact to produce the correct word form.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Examples of Unification-Based Approaches:

- ❖ **Head-Driven Phrase Structure Grammar (HPSG):** A type of grammar that relies heavily on unification for both syntax and morphology. In HPSG, words are described by complex feature structures, and morphological rules are defined through unification constraints.
- ❖ **Lexical Functional Grammar (LFG):** Another formal grammar that uses unification to manage the interactions between different linguistic levels, including morphology.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Consider the English word "walked":

- **Feature Structures:**

- The root "walk" might have features like:
 - ROOT: "walk"
 - POS: verb
- The past tense suffix "-ed" might have features like:
 - TENSE: past
 - POS: suffix

- **Unification:**

- The unification process would combine these feature structures to produce a complete feature structure for "walked":
 - ROOT: "walk"
 - POS: verb
 - TENSE: past

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Functional Morphology

- ❖ Functional morphology defines its models using principles of functional programming and type theory.
- ❖ It treats morphological operations and processes as pure mathematical functions and organizes the linguistic as well as abstract elements of a model into distinct types of values and type classes.
- ❖ Though functional morphology is not limited to modelling particular types of morphologies in human languages, it is especially useful for fusional morphologies.
- ❖ Linguistic notions like paradigms, rules and exceptions, grammatical categories and parameters, lexemes, morphemes, and morphs can be represented intuitively(without conscious reasoning; instinctively) and succinctly(in a brief and clearly expressed manner) in this approach.
- ❖ Functional morphology implementations are intended to be reused as programming libraries capable of handling the complete morphology of a language and to be incorporated into various kinds of applications.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- ❖ Morphological parsing is just one usage of the system, the others being morphological generation, lexicon browsing, and so on.
- ❖ we can describe inflection I , derivation D , and lookup L as functions of these generic type

$$I :: \text{lexeme} \rightarrow \{\text{parameter}\} \rightarrow \{\text{form}\} \quad (1.3)$$

$$D :: \text{lexeme} \rightarrow \{\text{parameter}\} \rightarrow \{\text{lexeme}\} \quad (1.4)$$

$$L :: \text{content} \rightarrow \{\text{lexeme}\} \quad (1.5)$$

Many functional morphology implementations are embedded in a general-purpose programming language, which gives programmers more freedom with advanced programming techniques and allows them to develop full-featured, real-world applications for their models.

Function Morphology

- ❖ It is not a widely recognized term in linguistic theory, but it might refer to a conceptual or theoretical approach where morphological processes are understood and described in terms of functions, particularly in computational models of language. If this is what you're referring to, it can be interpreted in a couple of ways:

1) Morphological Functions:

In this interpretation, morphology is seen as serving specific functions within a language, such as creating new word forms, marking grammatical categories (like tense, number, or case), or generating derivational forms. Each morphological process (e.g., affixation, compounding) has a specific function in language structure and communication.

2) Morphology in Functional Grammar:

In functional grammar theories, such as those proposed by Simon Dik (Functional Grammar) or in more general functionalist approaches, morphology is analyzed in terms of its role in achieving communicative purposes. Here, morphological markers are seen as tools that help speakers express meanings, organize discourse, or maintain cohesion.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

3) Functional Programming Approaches to Morphology:

- ❖ In computational linguistics, the term might refer to using **functional programming** principles to model and implement morphological processes. In this context, "functions" are programming constructs that take inputs (e.g., root words, grammatical features) and produce outputs (e.g., inflected forms).

• **Morphological Functions** could be written as functions in a programming language that map morphemes and features to the appropriate word forms. For example:

```
pluralize("cat") -> "cats"  
past_tense("run") -> "ran"
```


Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

```
def past_tense(verb):  
    if verb == "run":  
        return "ran"  
    else:  
        return verb + "ed"  
  
def conjugate(verb, tense):  
    if tense == "past":  
        return past_tense(verb)  
    elif tense == "present":  
        return verb + "s" if verb.endswith("e") else verb + "es"  
    # Add more cases for other tenses as needed
```



Lecture-#5

Natural Language Processing

A8706

Introduction

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

2. Finding structure of Documents

1. Introduction

- Finding the structure of documents is a crucial task in document analysis and information retrieval. Understanding the structure of a document allows for effective organization, navigation, summarization, and retrieval of information within the document. Here are some common techniques and approaches for finding the structure of documents:

Document Preprocessing:

Tokenization: Divide the text into individual tokens (words, phrases, sentences) to identify the basic units of text.

Sentence Segmentation: Split the text into sentences to analyze the document at a more granular level.

Headings and Sections:

Text Analysis: Use natural language processing (NLP) techniques to analyze the document's text and identify headings, subheadings, and section titles. These are often characterized by font size, style, or markup tags (e.g., HTML <h1>, <h2>).

Table of Contents: Extract the table of contents if available in the document.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Tokenization

- **Sentence Tokenization:** Splitting the document into individual sentences.
- **Word Tokenization:** Splitting sentences into individual words or tokens.

Part-of-Speech Tagging (POS Tagging)

- Assigning parts of speech (e.g., noun, verb, adjective) to each word in a sentence. This helps in understanding the grammatical structure of the sentence.

Dependency Parsing

- Analyzing the grammatical structure of a sentence by identifying the dependencies between words. This method identifies which words are the subject, object, and how they relate to the main verb.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Constituency Parsing

- Breaking down a sentence into sub-phrases or constituents, such as noun phrases (NP) and verb phrases (VP). This method helps in understanding how different parts of a sentence group together.

Named Entity Recognition (NER)

- Identifying entities (e.g., names of people, organizations, dates) in the text. NER helps in extracting important information and understanding the document's context.

Syntactic Tree Structures

- Representing the sentence structure as a tree where each node represents a syntactic category (e.g., NP, VP) and leaves represent words. Trees can be generated using parsing techniques like constituency or dependency parsing.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Coreference Resolution

- Identifying when different words in a document refer to the same entity (e.g., "John" and "he"). This helps in maintaining coherence across the document.

Semantic Role Labeling (SRL)

- Assigning roles to words or phrases in a sentence to determine who did what to whom. SRL helps in understanding the sentence's meaning beyond just syntactic structure.

Text Structure Analysis

- For documents, analyzing the overall structure such as headings, paragraphs, and sections. This includes understanding the hierarchical nature of the document, identifying titles, abstracts, and main content sections.

Discourse Parsing

- Analyzing the structure of entire texts by identifying how sentences and paragraphs are connected logically. This includes understanding the rhetorical relations between different parts of the text.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Tools and Libraries

NLTK (Natural Language Toolkit): A Python library for working with human language data.

spaCy: A powerful NLP library for parsing, tagging, and entity recognition.

Stanford NLP: A suite of NLP tools that includes parsers, POS taggers, and more.

CoreNLP: A tool from Stanford for natural language processing, including parsing and sentiment analysis.

OpenNLP: An Apache project that provides machine learning-based NLP tools.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

1. Sentence Boundary Detection

❖ **Sentence Boundary Detection (SBD)**, also known as sentence segmentation or sentence tokenization, is the process of identifying the boundaries between sentences within a block of text. Accurate sentence boundary detection is a crucial preprocessing step in natural language processing (NLP) tasks, as it allows you to analyze and process text at the sentence level.

❖ In written text in English and some other languages, the beginning of a sentence is usually marked with an uppercase letter, and the end of a sentence is explicitly marked with a period(.), a question mark(?), an exclamation mark(!), or another type of punctuation.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- **Key Challenges in Sentence Boundary Detection**
- **Punctuation Ambiguity:**
 - Periods:** The most common sentence-ending punctuation mark can also appear in abbreviations (e.g., "Dr."), numbers (e.g., "3.14", "S.No."), or within quotes.
 - Ellipses:** "..." can indicate an unfinished thought or continuation rather than the end of a sentence.
 - Question and Exclamation Marks:** These are more reliable but still can be ambiguous in some contexts.
- **Abbreviations:**
 - Abbreviations (e.g., "e.g.," "i.e.," "Mr.") can falsely signal a sentence boundary when they appear at the end of a word.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Quotation Marks:

Sentences ending with a quote can confuse the system as it may not be clear if the period or other punctuation inside the quotes ends the sentence or if the sentence continues outside the quotes.

Multiple Sentences in One Line:

Sometimes, multiple sentences are written in a single line without clear markers, making it harder to detect boundaries.

Languages with No Clear Sentence Boundaries:

In languages like Chinese or Japanese, where there is no punctuation to clearly mark sentence boundaries, SBD is particularly challenging.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Rule-Based Approaches:

Punctuation Rules: One of the simplest methods is to split sentences at punctuation marks such as periods, question marks, and exclamation points. However, this method may lead to incorrect splits in cases where periods are used in abbreviations or numbers.

Abbreviation Detection: To address the issue of abbreviations, you can create rules to detect common abbreviations and avoid splitting sentences on periods within them.

Quotation Marks: Splitting sentences at quotation marks (e.g., ".", "?", or "!") outside of quotation marks can be effective for many texts.

Machine Learning Approaches:

Supervised Learning: You can train a supervised machine learning model (e.g., a binary classifier) to predict whether a punctuation mark indicates the end of a sentence. Features might include the punctuation mark itself, surrounding words, and contextual information.

Conditional Random Fields (CRF): CRFs are a type of probabilistic graphical model that can be used for sequence labeling tasks like SBD. They consider the entire sequence of tokens when making segmentation decisions.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Deep Learning Approaches:

Recurrent Neural Networks (RNNs): RNNs, particularly Long Short-Term Memory (LSTM) or Gated Recurrent Unit (GRU) variants, can be used to model sequences of words and predict sentence boundaries based on the context.

Transformer Models: State-of-the-art models like BERT and GPT can be fine-tuned for sentence boundary detection as they have a strong understanding of contextual information.

Lexical Clues:

Capitalization: Sentences often start with a capital letter. You can use capitalization rules to detect sentence boundaries.

Numbers: Sentences often start after numbers, especially when followed by punctuation.

Language-Specific Approaches:

Languages may have specific grammatical rules that can be leveraged for SBD. **For example**, in languages like Japanese and Chinese, sentence boundaries are not indicated by spaces but by specific characters.

Third-Party Tools:

There are many NLP libraries and tools (e.g., NLTK, spaCy) that include pre-trained models and functions for sentence boundary detection. These tools often combine rule-based and machine learning techniques.

Evaluation:

Once you implement an SBD algorithm, it's important to evaluate its performance using metrics like precision, recall, and F1-score on a labeled dataset to ensure its accuracy.

Handling Edge Cases:

Consider handling edge cases, such as abbreviations, ellipses, and non-standard punctuation, to improve the robustness of your SBD algorithm.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

2. Topic Boundary Detection

- Topic boundary detection is a natural language processing (NLP) task that involves identifying points within a text where the topic or subject matter changes.
- Detecting topic boundaries is essential for various NLP applications, such as text summarization, content segmentation, document classification, and information retrieval. Here are some techniques and approaches for topic boundary detection:

Keyword-Based Approaches:

Identify keywords or key phrases that indicate a change in topic. These keywords can be domain-specific or general words that typically signal topic shifts (e.g., "however," "on the other hand," "in conclusion").

Term Frequency and Inverse Document Frequency (TF-IDF):

Calculate TF-IDF scores for terms in the document. A sudden increase in the TF-IDF score of a term may signal a topic boundary. A drop in TF-IDF values can also be indicative of a topic shift.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

2. Topic Boundary Detection

Cosine Similarity:

Measure the cosine similarity between document segments or paragraphs. A significant drop in similarity between consecutive segments may indicate a topic change.

Supervised Learning:

Train a supervised machine learning model, such as a binary classifier, to predict topic boundaries based on labeled data. Features for the model might include linguistic cues, such as punctuation, sentence length, and word usage.

Clustering:

Use clustering algorithms (e.g., k-means clustering) to group similar paragraphs or sentences together. Changes in cluster assignments can indicate topic boundaries.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

•How TextTiling Works

•Tokenization:

–The text is first tokenized into words, and then into blocks (windows) of words.

•Lexical Scoring:

–The tokenizer calculates a lexical similarity score between adjacent blocks. If the similarity drops significantly, it's assumed that a topic boundary exists.

•Boundary Detection:

–The algorithm identifies where the largest drops in similarity occur and marks those points as potential topic boundaries.



Lecture-#6

Natural Language Processing

A8706

Introduction

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

2.2. Methods

- ❖ Sentence segmentation and topic segmentation have been considered as a **boundary classification problem**.
- ❖ Given a boundary candidate(between two word tokens for sentence segmentation and between two sentences for topic segmentation), the goal is to predict whether or not the candidate is an actual boundary (sentence or topic boundary).
- ❖ Formally, let $\mathbf{x} \in \mathcal{X}$ be the **vector of features** (the observation) associated with a candidate and $\mathbf{y} \in \mathcal{Y}$ be the **label** predicted for that candidate.
- ❖ The label $\mathbf{y}$ can be **$\mathbf{b}$ for boundary** and **$\mathbf{b}'$ for non boundary**.
 - ❖ Classification problem: given a set of training examples $(\mathbf{x}, \mathbf{y})_{\text{train}}$, find a function that will assign the most accurate possible label $\mathbf{y}$ of unseen examples $\mathbf{x}$ unseen.
 - ❖ Alternatively to the binary classification problem, it is possible to model boundary types using finer-grained categories.
 - ❖ For segmentation in text be framed as a three-class problem: sentence boundary $\mathbf{b}^a$, without an abbreviation $\mathbf{b}^{a'}$ and abbreviation not as a boundary $\mathbf{b}^{-a}$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

It is also possible to see the candidate boundaries as a sequence and search for the sequence of boundary types $\hat{Y} = (\hat{y}_1, \dots, \hat{y}_n)$ that have the maximum probability given the candidate examples, $\mathbf{X} = (x_1, \dots, x_n)$:

$$\hat{Y} = \arg \max_y P(Y|X)$$

- We categorize the methods into local and sequence classification.
- Another categorization of methods is done according to the type of the machine learning algorithm: **generative versus discriminative**.
- Generative sequence models estimate the **joint distribution** of the observations $P(X, Y)$ (words, punctuation) and the labels (sentence boundary, topic boundary).
- Discriminative sequence models, however, **focus on features** that categorize the differences between the labelling of that example.

- $\hat{y}_i$: The predicted label for the input x_i .
- $P(y_i | x_i)$: The posterior probability of label y_i given the input x_i .
- $\arg \max_{y_i \in \mathcal{Y}}$: The operation that finds the value of y_i in the set of possible labels $\mathcal{Y}$ that maximizes the posterior probability.

The posterior probability is given by Bayes' theorem:

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y) \cdot P(y)}{P(\mathbf{x})}$$

Where:

- $P(y|\mathbf{x})$: The posterior probability of label y given the data $\mathbf{x}$.
- $P(\mathbf{x}|y)$: The likelihood, which is the probability of the data $\mathbf{x}$ given that the label is y .

Example: Spam Email Classification

Suppose you are working on a spam email classifier. The goal is to classify whether an email is "spam" or "not spam" based on its content.

1.Input (x_i): Features extracted from the email text, like the frequency of certain words (e.g., "free," "win," "money").

2.Possible Labels ($\mathcal{Y}$): The set of possible labels is $\mathcal{Y} = \{\text{"spam"}, \text{"not spam"}\}$.

3.Posterior Probability ($P(y_i|x_i)$): This is the probability that the email is either "spam" or "not spam" given the features.

The MAP estimate involves calculating the posterior probability for both "spam" and "not spam" labels and then choosing the label with the highest probability.

•For example, given an email with certain words, you might calculate:

- $P(\text{"spam"}|x_i) = 0.8$
- $P(\text{"not spam"}|x_i) = 0.2$

4.Prediction ($\hat{y}_i$): The classifier will predict "spam" because it has the higher posterior probability.

Thus, the formula $\hat{y}_i = \arg \max_{(y_i \in \mathcal{Y})} P(y_i|x_i)$ in this context would output "spam" as the predicted label.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Generative Sequence Classification Methods:

- ❖ Generative sequence classification methods are a class of machine learning techniques that combine elements of sequence generation and classification tasks.
- ❖ These methods aim to generate sequences while also assigning a class label or category to each sequence.
- ❖ This can be particularly useful in various natural language processing (NLP) and bioinformatics tasks, among others.
- ❖ Generative sequence classification methods find applications in various domains, including natural language processing, bioinformatics, speech recognition, and more.
- ❖ The choice of method depends on the specific task and dataset, as well as the available computational resources and desired model performance.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- A Hidden Markov Model (HMM) is a statistical model that represents systems where the state is partially observable. It's widely used in various applications such as speech recognition, natural language processing, and bioinformatics.

- **Components of an HMM**

- 1. States:**

- These are the possible conditions or situations that the system can be in. However, in an HMM, these states are hidden, meaning you cannot directly observe them.
- Example: In a speech recognition system, the states could represent different phonemes (basic units of sound).

- 2. Observations:**

- These are the data or evidence that we can observe and are influenced by the underlying hidden states.
- Example: In speech recognition, the observations could be the acoustic signals or features extracted from audio.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

3. Transition Probabilities (A):

- These probabilities represent the likelihood of moving from one state to another.

$$a_{ij} = P(S_{t+1} = j | S_t = i)$$

- is the probability of transitioning from state i to state j

4. Emission Probabilities (B):

- These are the probabilities of observing a particular observation given a specific state.

$$b_j(o_t) = P(O_t = o | S_t = j)$$

is the probability of observing o given the system is in state j.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

5. Initial Probabilities (π):

$$\pi_i = P(S_1 = i)$$

These are the probabilities of the system starting in a particular state.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- **Example: Part-of-Speech Tagging**

- Imagine you're working on tagging words in a sentence with their corresponding part-of-speech (POS) tags (e.g., noun, verb).
- **States:** The possible POS tags (e.g., Noun, Verb, Adjective).
- **Observations:** The words in the sentence.
- **Transition Probabilities:** The probability of moving from one POS tag to another.
- **Emission Probabilities:** The probability of a particular word being associated with a particular POS tag.
- **Initial Probabilities:** The probability distribution over POS tags for the first word in the sentence.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Generative Sequence Classification Methods:

Generative models, to find the conditional probability $P(Y|X)$, they estimate the prior probability $P(Y)$ and likelihood probability $P(X|Y)$ with the help of the training data and use the Bayes Theorem to calculate the posterior probability $P(Y|X)$:

$$\text{posterior} = \frac{\text{prior} \times \text{likelihood}}{\text{evidence}} \Rightarrow P(Y|X) = \frac{P(Y) \cdot P(X|Y)}{P(X)}$$

$P(X)$ in the denominator is dropped because it is fixed for different Y and hence does not change the argument of max. $P(X|Y)$, and $P(Y)$ can be estimated as

$$P(X|Y) = \prod_{i=1}^n P(\mathbf{x}_i | y_1, \dots, y_i)$$

$$P(Y) = \prod_{i=1}^n P(y_i | y_1, \dots, y_{i-1})$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Simplifying assumptions can be made to make the computation of these probabilities tractable:

$$P(\mathbf{x}_i | y_1, \dots, y_i) \approx P(\mathbf{x}_i | y_i)$$

and a bigram model can be assumed for modeling output categories:

$$P(y_i | y_1, \dots, y_{i-1}) \approx P(y_i | y_{i-1})$$

Finding the Structure of Words and Documents

Model for the two-class problem, for example *nonboundary* (NB) and *sentence boundary* (SB) for sentence segmentation.

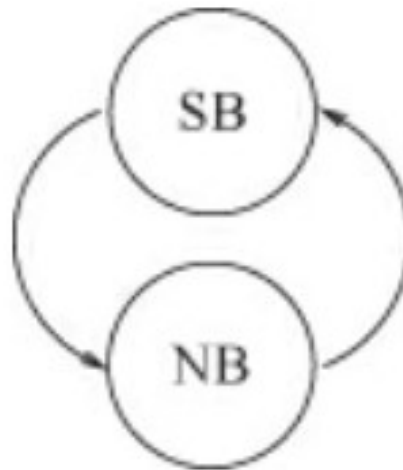


Fig: Conceptual hidden Markov model for segmentation with two states: one for segment boundaries, one for others

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

<i>Emitted Words</i>	...	people	are	dead	few	pictures	...
<i>State Sequence</i>	...	NB	NB	SB	NB	NB	...

Sentence segmentation with simple two-state Markov model

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Mr. Smith went to Washington. He had a meeting at 10 a.m. It was rescheduled.

Annotated with SB and NB:

1. "Mr. Smith went to Washington."

1. "Mr." → **NB** (Non-boundary, because "Mr." is an abbreviation, not the end of a sentence)
2. "Washington." → **SB** (Sentence Boundary, marking the end of a sentence)

2. "He had a meeting at 10 a.m."

1. "a.m." → **NB** (Non-boundary, because "a.m." is an abbreviation, not the end of a sentence)
2. "a.m." → **SB** (Sentence Boundary, marking the end of a sentence after "a.m.")

3. "It was rescheduled."

1. "rescheduled." → **SB** (Sentence Boundary, marking the end of a sentence)

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Mr. Smith went to Washington. He had a meeting at 10 a.m. It was rescheduled.

- **Non-boundary (NB):** This is where a period (or other punctuation) does not indicate the end of a sentence. Common examples include abbreviations like "Mr.," "Dr.," "a.m.," or "etc."

- **Examples:**

- "Mr. Smith" → **NB**

- "10 a.m." → **NB**

- **Sentence Boundary (SB):** This is where a period (or other punctuation) actually marks the end of a sentence.

- **Examples:**

- "Washington." → **SB**

- "a.m." (when followed by a capitalized word or new sentence) → **SB**

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Generative models are considered a class of statistical models that can generate new data instances. These models are used in unsupervised machine learning as a means to perform tasks such as:

- ❖ Probability and Likelihood estimation,
- ❖ Modeling data points
- ❖ To describe the phenomenon in data,
- ❖ To distinguish between classes based on these probabilities.

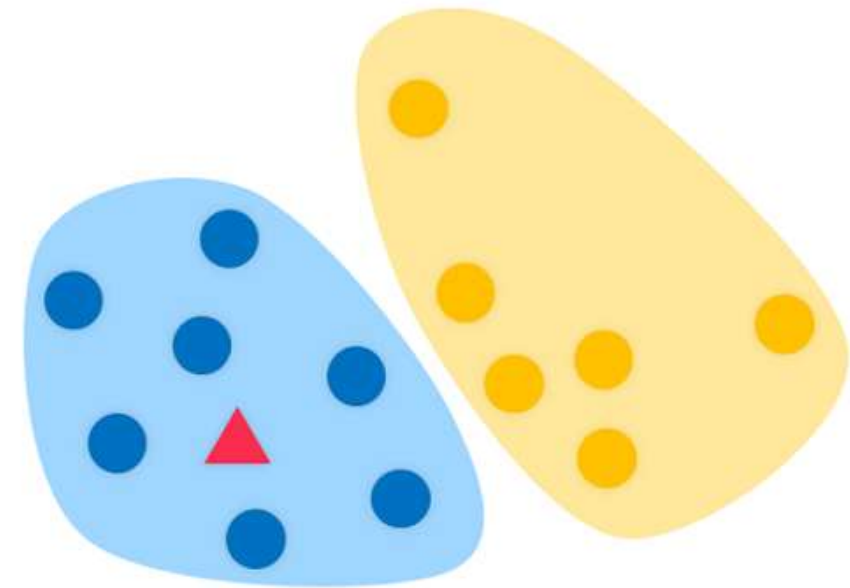
Since these models often rely on the Bayes theorem to find the joint probability, generative models can tackle a more complex task than analogous discriminative models.

Finding the Structure of Words and Documents

❖ The Generative approach focuses on the distribution of individual classes in a dataset, and the learning algorithms tend to model the underlying patterns or distribution of the data points (e.g., gaussian). These models use the concept of joint probability and create instances where a given **feature (x)** or input and the desired output or **label (y)** exist simultaneously.

❖ These models use **probability estimates** and **likelihood** to model data points and differentiate between different class labels present in a dataset. Unlike discriminative models, these models can also generate new data points.

Generative



Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Training generative classifiers involve estimating a function $f: X \rightarrow Y$, or probability $P(Y|X)$:

- ❖ Assume some functional form for the probabilities such as $P(Y)$, $P(X|Y)$
- ❖ With the help of training data, we estimate the parameters of $P(X|Y)$, $P(Y)$
- ❖ Use the Bayes theorem to calculate the posterior probability $P(Y|X)$

Examples of Generative Models

- ❖ Naïve Bayes
- ❖ Bayesian networks
- ❖ Markov random fields
- ❖ Hidden Markov Models (HMMs)
- ❖ Latent Dirichlet Allocation (LDA)
- ❖ Generative Adversarial Networks (GANs)
- ❖ Autoregressive Model

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Discriminative Local Classification Methods:

- ❖ Discriminative local classification methods are a class of machine learning techniques that focus on making decisions or predictions for individual data points within a **larger dataset**.
- ❖ These methods aim to classify data points into specific categories or classes based on their features while considering the local context around each data point.
- ❖ The discriminative model refers to a class of models used in **Statistical Classification**, mainly used for supervised machine learning.
- ❖ These types of models are also known as **conditional models** since they learn the boundaries between classes or labels in a dataset.
- ❖ These discriminative local classification methods are used in a wide range of applications, including **image classification**, **text classification**, **anomaly detection**, and more.
- ❖ The choice of method depends on the nature of the data, the problem at hand, and the trade-offs between **model complexity** and **performance**.

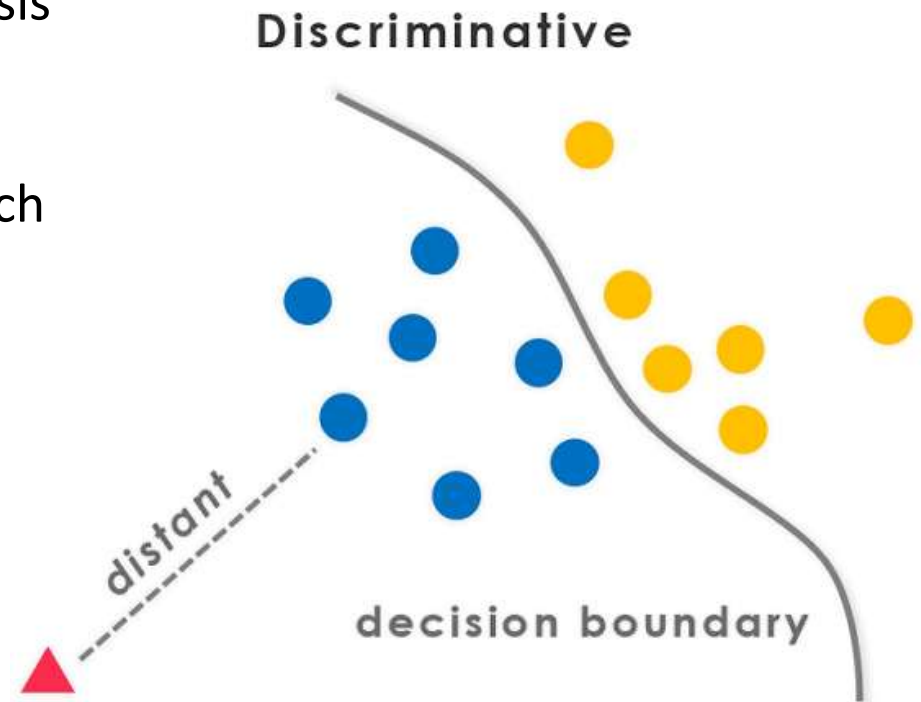
Finding the Structure of Words and Documents

Training discriminative classifiers or discriminant analysis involves estimating a function $f: X \rightarrow Y$, or probability $P(Y|X)$

- ❖ Assume some functional form for the probability, such as $P(Y|X)$
- ❖ With the help of training data, we estimate the parameters of $P(Y|X)$

Examples of Discriminative Models

- ❖ Logistic regression
- ❖ Support vector machines(SVMs)
- ❖ Traditional neural networks
- ❖ Nearest neighbor
- ❖ Conditional Random Fields (CRFs)
- ❖ Decision Trees and Random Forest



Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

•Local Classification:

- In structured prediction tasks, such as **part-of-speech tagging** or **named entity recognition**, a local classifier is applied to each element (**word, token**) in the sequence to make a prediction.
- The prediction for each element is made independently, often based on a window of surrounding elements (the local context).

•Discriminative Models:

- Discriminative models are those that model the decision boundary directly. Instead of modeling the joint probability of the inputs and outputs, they model the **conditional probability $P(y|x)$**

•Structured Prediction:

- Structured prediction refers to tasks where the output is not a single label, but a structured object like a sequence or a tree. Examples include **sequence labeling tasks**, **parsing**, and **machine translation**.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

•Advantages of Discriminative Local Classification

- **Flexibility:** Discriminative models can directly incorporate a variety of features, including non-independent features, making them more flexible than generative models.
- **Efficiency:** Local classifiers can be computationally efficient because they make predictions based on local information, rather than modeling the entire sequence globally.
- **Scalability:** These methods can scale well to large datasets and complex tasks by focusing on local decisions.

•Disadvantages

- **Error Propagation:** Since decisions are made locally, errors made early in the sequence can propagate, leading to errors later in the sequence.
- **Lack of Global Context:** These methods may miss global dependencies that could be captured by global models or structured prediction approaches like CRFs.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

The quick brown fox jumps over the lazy dog.

Steps Involved

1. Feature Extraction

For each word in the sentence, we extract features from a window of words around it. These features could include:

- The word itself
- Previous and next words
- Capitalization (is the word capitalized?)
- Suffixes or prefixes of the word
- Part-of-speech tags of previous words (if already predicted)
- Position in the sentence

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

The quick brown fox jumps over the lazy dog.

2. Local Classification

We use a discriminative model (e.g., logistic regression, SVM, or a neural network) to predict the POS tag for each word based on the extracted features.

Let's consider the first few words:

Word: "The"

Features:

Current word: "The"

Next word: "quick"

Capitalization: Yes

Prefixes/Suffixes: "Th-", "-he"

Word: "quick"

Features:

Previous word: "The"

Current word: "quick"

Next word: "brown"

Capitalization: No

Prefixes/Suffixes: "qui-", "-ick"

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

The quick brown fox jumps over the lazy dog.

3. Prediction

The model makes a prediction for each word:

"The" → DET (Determiner)

"quick" → ADJ (Adjective)

"brown" → ADJ (Adjective)

"fox" → NOUN (Noun)

"jumps" → VERB (Verb)

"over" → PREP (Preposition)

"the" → DET (Determiner)

"lazy" → ADJ (Adjective)

"dog" → NOUN (Noun)

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Two Methods for computing the similarity scores

- block comparison
- vocabulary introduction

Given two blocks, b_1 and b_2 , each having k tokens (sentences or paragraphs), the similarity (or topical cohesion) score is computed by the formula:

$$\frac{\sum_t w_{t,b_1} \cdot w_{t,b_2}}{\sqrt{\sum_t w_{t,b_1}^2 \sum_t w_{t,b_2}^2}}$$

where $w_{t,b}$ is the weight assigned to term t in block b . The weights can be binary or may be computed using other information retrieval-based metrics such as term frequency.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Assigns a score to a token-sequence gap on the basis of how many new words are seen in the interval in which it is the midpoint. Similar to the block comparison formulation, given two consecutive blocks, b_1 and b_2 , of equal number of words, w , the topical cohesion score is computed with the following formula:

$$\frac{NumNewTerms(b_1) + NumNewTerms(b_2)}{2 \times w}$$

where $NumNewTerms(b)$ returns the number of terms in block b , seen for the first time in text.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Discriminative Sequence Classification Methods:

- ❖ In segmentation tasks, the sentence or topic decision for a given example(word, sentence, paragraph) highly depends on the decision for the examples in its vicinity.
- ❖ Discriminative sequence classification methods are in general extensions of local discriminative models with additional decoding stages that find the best assignment of labels by looking at neighboring decisions to label.
- ❖ Conditional random fields(CRFs) are extension of maximum entropy, SVM struct is an extension of SVM, and maximum margin Markov networks(M³N) are extensions of HMM.
- ❖ CRFs are a class of log-linear models for labelling structures.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- They Predict sentence or topic boundaries independently, CRFs can oversee the whole sequence of boundary Hypotheses to make their decisions.
- Formally, they model the conditional probability of a sequence of boundary labels ($Y = y_1, \dots, y_n$) given the sequence of feature sets extracted from the context in which they occur ($X = \mathbf{x}_1, \dots, \mathbf{x}_n$).

$$P(Y|X) \sim \frac{1}{Z(X)} \exp \left(\sum_{t=1}^n \sum_{i=1}^m \lambda_i f_i(y_{t-1}, y_t, y_t) \right)$$

$$Z(X) = \sum_Y \exp \left(\sum_{t=1}^n \sum_{i=1}^m \lambda_i f_i(y_{t-1}, y_t, y_t) \right)$$

where $f_i(\cdot)$ are feature functions of the observations and a clique of labels, and λ_i are the corresponding weights. $Z(\cdot)$ is a normalization function dependent only on the observations.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

1. Sequence Classification:

- Unlike local classification, which makes predictions independently for each element, sequence classification considers the entire sequence of inputs when making predictions.
- The goal is to assign a label to each element in the sequence, taking into account the dependencies between neighbouring elements.

2. Discriminative Models:

- Discriminative models focus on directly estimating the conditional probability $P(y \mid x)$ where x is the sequence of inputs and y is the sequence of labels.
- These models are trained to optimize the decision boundary between different classes based on the input features.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

3. Common Discriminative Sequence Models:

- **Conditional Random Fields (CRFs):** One of the most popular discriminative models for sequence classification. CRFs model the conditional probability of the entire sequence of labels given the sequence of inputs.
- **Structured Support Vector Machines (SVMs):** These extend the SVM framework to handle structured outputs, including sequences.
- **Recurrent Neural Networks (RNNs):** RNNs, including Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), are powerful models for handling sequences because they can maintain a memory of previous inputs in the sequence.
- **Transformer-based Models:** Models like BERT and GPT, which rely on the transformer architecture, are also discriminative and can handle sequence prediction by leveraging attention mechanisms to capture long-range dependencies.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

John Doe works at OpenAI in San Francisco.

Using Conditional Random Fields (CRFs)

1. Feature Extraction

For each word in the sentence, extract features such as:

- The word itself.
- Word shape (capitalization, digits).
- Part-of-speech (POS) tags.
- Contextual words (words before and after the current word).

2. Modeling with CRFs

- Sequence Model:** CRFs will model the conditional probability of the sequence of labels (e.g., "B-PER", "I-PER", "O", "B-ORG", "O", "B-LOC", "I-LOC") given the input sequence (the words in the sentence).
- Dependency Capture:** CRFs capture dependencies between neighboring labels, meaning the model understands that if a word is labeled "B-PER" (beginning of a person's name), the next word is likely "I-PER" (inside a person's name) if it's part of the same entity.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

John Doe works at OpenAI in San Francisco.

3. Training

- The CRF model is trained on annotated data, where each word in the sequence is labeled with its corresponding named entity tag.
- During training, the model learns to maximize the conditional likelihood of the correct sequence of labels given the input sequence.

4. Inference

- Given a new sentence, the CRF model will predict the most likely sequence of tags that fits the entire sequence of words, taking into account both the individual word features and the transition probabilities between tags.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Advantages of Discriminative Sequence Models

- **Global Sequence Consideration:** These models consider the entire sequence, which helps capture dependencies between labels that local classification methods might miss.
- **Flexibility in Feature Usage:** Discriminative models can directly incorporate complex and rich features, including non-independent features.
- **Accuracy:** By focusing on the decision boundary, discriminative models often provide better performance in practice, especially in complex tasks.

Disadvantages

- **Complexity:** These models can be more complex to train and require more computational resources, especially with large datasets.
- **Error Sensitivity:** While these models capture dependencies, they can also be sensitive to errors in the input features or label dependencies, potentially leading to cascading errors.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- Nonsequential discriminative classification algorithms typically ignore the context, which is critical for the segmentation task.
- While we may add context as a feature or simply use CRFs, which inherently consider context, these approaches are suboptimal when dealing with real-valued features, such as pause duration or pitch range.
- Most earlier studies simply tackled this problem by binning the feature space either manually or automatically

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- The main idea is to use the posterior probabilities, $P_c(y_i|\mathbf{x}_i)$, for each boundary candidate, obtained from the other classifiers, such as boosting or CRF, by simply converting them to state observation likelihoods by dividing to their priors following the well-known Bayes rule:

$$\operatorname{argmax}_{y_i} \frac{P_c(y_i|\mathbf{x}_i)}{P(y_i)} = \operatorname{argmax}_{y_i} P(\mathbf{x}_i|y_i)$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- Applying the Viterbi algorithm to the HMM then returns the most likely segmentation. To handle dynamic ranges of state transition probabilities and observation likelihoods, a weighting scheme as is usually described in the literature can be applied:

$$\operatorname{argmax}_{y_i} P_c(\mathbf{x}_i|y_i)^\alpha \times P(y_i)^\beta$$

where $P(y)$ is estimated by the HELM, and α and β are optimized using a held-out set.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- Applying the Viterbi algorithm to the HMM then returns the most likely segmentation. To handle dynamic ranges of state transition probabilities and observation likelihoods, a weighting scheme as is usually described in the literature can be applied:

$$\operatorname{argmax}_{y_i} P_c(\mathbf{x}_i|y_i)^\alpha \times P(y_i)^\beta$$

where $P(y)$ is estimated by the HELM, and α and β are optimized using a held-out set.

- The **hold-out method** involves splitting the data into multiple parts and **using** one part for training the model and the rest for validating and testing it.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- **HELM** (the hidden event language model)

- In this model, one state is reserved for each boundary token, *SB* and *NB*, and the rest of the states are for generating words. To ease the computation, an imaginary token is inserted between all consecutive words in case the word preceding the boundary is not part of a disfluency.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

- ❖ Hybrid approaches in Natural Language Processing (NLP) classifications refer to techniques that combine multiple methods or models to improve the performance of NLP classification tasks.
- ❖ These methods often leverage the strengths of different approaches to address the limitations of individual models. Here are some common hybrid approaches in NLP classifications:

Ensemble Methods:

- ❖ Ensemble methods combine predictions from multiple base models to make a final classification decision.
- ❖ Bagging (Bootstrap Aggregating) methods like Random Forests combine multiple decision trees.
- ❖ Boosting methods like AdaBoost and Gradient Boosting combine weak learners to create a strong classifier.
- ❖ These ensemble methods can improve classification accuracy and robustness.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

Feature Combination:

- ❖ Hybrid models can combine features from different sources or representations to enhance classification performance.
- ❖ For example, combining word embeddings with hand-crafted features or syntactic features can capture both semantic and structural information.

Multi-Modal Fusion:

- ❖ In some NLP tasks, information comes from multiple modalities, such as text, images, or audio.
- ❖ Hybrid models can fuse information from these modalities to make better classifications. For instance, combining textual and visual information for sentiment analysis of multimedia content.

Neural Network Ensembles:

- ❖ Combining multiple neural networks with different architectures or initialization weights can lead to improved performance.
- ❖ Techniques like stacking or blending neural network predictions can be effective in hybrid approaches.

Transfer Learning and Pretraining:

- ❖ Transfer learning models like BERT, GPT, and RoBERTa have become popular in NLP.
- ❖ Hybrid models can fine-tune these pretrained models for specific classification tasks, achieving state-of-the-art results with less training data.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Hybrid Approaches:

Rule-Based and Machine Learning Hybrid:

- ❖ Combining rule-based systems with machine learning models can leverage domain knowledge while benefiting from data-driven approaches.
- ❖ For instance, using regular expressions or handcrafted rules to preprocess text data before feeding it into a machine learning classifier.

Hierarchical Approaches:

- ❖ Hybrid models can use a hierarchy of classifiers, where higher-level classifiers make decisions based on the outputs of lower-level classifiers.
- ❖ This approach is useful for complex classification tasks, where different aspects of the problem can be tackled independently.

Sequential and Tree-Based Models:

- ❖ Hybrid models can combine sequential models like Recurrent Neural Networks (RNNs) with tree-based models like Random Forests or Gradient Boosting.
- ❖ This approach can be useful for problems that involve both sequential and non-sequential information.
- Hybrid approaches in NLP classifications are often problem-specific and depend on the nature of the data and the task. They offer flexibility and the potential for significant performance improvements by leveraging the complementary strengths of different techniques and models.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Extensions for Global Modeling for Sentence Segmentation:

- ❖ In Natural Language Processing (NLP), sentence segmentation, also known as sentence boundary detection or sentence tokenization, is the process of identifying and splitting a text into individual sentences.
- ❖ While basic sentence segmentation methods often rely on simple heuristics or punctuation cues, more advanced techniques employ global modeling and extensions to improve accuracy and handle complex cases.
- ❖ These extensions for global modeling in sentence segmentation aim to handle various linguistic complexities, including abbreviations, quotations, nested structures, and different writing styles.
- ❖ The choice of method and extension depends on the specific requirements of the NLP task and the characteristics of the text data being processed.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Complexity of the Approaches

- ❖ In Natural Language Processing (NLP), the complexity of approaches refers to the computational, algorithmic, and resource requirements associated with different methods and models used for various NLP tasks.
- ❖ The complexity can vary significantly depending on the specific approach, the size of the dataset, the complexity of the language, and the available computational resources.

Model Architecture:

The choice of model architecture plays a significant role in complexity. For example, simple models like logistic regression have lower computational demands compared to deep neural networks such as Transformers.

Model Size:

Larger models with more parameters tend to be more complex. Pretrained models like GPT-3 and BERT, which have hundreds of millions or even billions of parameters, require substantial computational resources.

Training Data Size:

The amount of training data affects the complexity of model training. Larger datasets often require more computational power and time to train effectively.

Hyper parameter Tuning:

Finding the optimal hyper parameters for a model can involve a time-consuming and computationally intensive search, especially for complex models.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Tokenization and Sequence Length:

Tokenization, or splitting text into smaller units (e.g., words or subwords), can impact model complexity. Handling long sequences may require special techniques or models designed for long-range dependencies.

Data Preprocessing:

Data preprocessing steps such as text cleaning, feature extraction, and handling imbalanced datasets can add complexity to an NLP pipeline.

Fine-Tuning:

Fine-tuning pretrained models for specific tasks can be complex, as it involves adjusting model weights on a task-specific dataset while retaining knowledge from the pretrained model.

Decoding:

In sequence generation tasks like text summarization or machine translation, decoding the output sequence can be computationally intensive, especially for beam search or sampling-based methods.

Parallelization:

Parallelizing model training and inference across multiple GPUs or TPUs can mitigate complexity by reducing the time required for computation.

Resource Requirements:

Complex models often demand powerful hardware resources, including GPUs or TPUs, to achieve reasonable training and inference times.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Interpretability:

As models become more complex, their interpretability may decrease. Understanding and interpreting the decisions made by complex models can be challenging.

Scaling Up:

To achieve state-of-the-art performance, researchers often resort to scaling up models, which can lead to increased complexity and resource requirements.

Regularization:

To prevent overfitting in complex models, regularization techniques may be needed, adding complexity to the training process.

Deployment Considerations:

Deploying complex NLP models in real-world applications may require optimizations for latency, memory usage, and scalability.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Performance of approaches:

- ❖ The performance of approaches in Natural Language Processing (NLP) can vary significantly depending on the specific task, the choice of algorithms, models, and data.
- ❖ NLP tasks encompass a wide range of applications, and performance is typically evaluated using various metrics tailored to each task.
- ❖ Here are some key performance metrics and factors influencing the performance of NLP approaches:
 - ❖ **Accuracy:** Accuracy measures the proportion of correctly predicted instances in classification tasks. While accuracy is a common metric, it may not be appropriate for imbalanced datasets.
 - ❖ **Precision, Recall, and F1-Score:** Precision measures the fraction of true positives among predicted positives, while recall measures the fraction of true positives among actual positives. The F1-score is the harmonic mean of precision and recall, providing a balanced measure of performance.
 - ❖ **Mean Squared Error (MSE):** MSE is often used to evaluate regression tasks, measuring the average squared difference between predicted and actual values.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

1. Common Evaluation Metrics

- **Error Rate:**
 - The ratio of the number of errors to the total number of examples.
- **F-measure (or F1 score):**
 - The harmonic mean of **precision** and **recall**:
 - **Recall**: Proportion of actual sentence boundaries correctly identified.
 - **Precision**: Proportion of identified boundaries that are correct.
- **NIST Error Rate:**
 - Number of incorrectly labeled candidates divided by actual boundaries.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

2. Performance in Text

- **Dataset:** Wall Street Journal Corpus (~27,000 sentences).
- **Results:**
 - **Mikheev's rule-based system:**
 - Error rate: **1.41%**
 - **Adding an abbreviation list:**
 - Error rate drops to **0.45%**
 - **Combining with a supervised classifier (using POS tags):**
 - Error rate further decreases to **0.31%**
 - **Gillick's SVM-based system (no handcrafted rules or abbreviation list):**
 - Error rate: **0.25%**
- **Implication:**
 - Even small error rates are important, as mistakes in sentence segmentation affect all downstream NLP tasks and user-facing outputs.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

3. Performance in Speech

• Mandarin Broadcast News (TDT4 Corpus, Doss et al.):

- **MaxEnt classifier:**
 - F-measure: **69.1%**
- **Adaboost:**
 - F-measure: **72.6%**
- **SVMs:**
 - F-measure: **72.7%**
- **Combination via logistic regression:**
 - Can further improve results.

• Turkish Broadcast News (Guz et al.):

- **HELM:**
 - F-measure: **78.2%**
- **fHELM with morphology features:**
 - F-measure: **86.2%**

• English Broadcast News (Zimmerman et al.):

- **Adaboost + HELM:**
 - F-measure: **67.3%**
- **Adaboost alone:**
 - F-measure: **65.5%**

Adaboost:

F-measure: **86.9%**

CRFs:

F-measure: **89.1%**

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

- Precision is a common metric used to evaluate the performance of classification models, especially in scenarios where false positives are particularly undesirable.

- **Formula for Precision**

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **True Positives (TP):** Instances correctly predicted as positive.

- **False Positives (FP):** Instances incorrectly predicted as positive.

- **Steps to Calculate Precision**

- **Identify True Positives (TP):** Count how many positive observations were correctly predicted.
- **Identify False Positives (FP):** Count how many negative observations were incorrectly predicted as positive.

- **Apply the formula:** Divide the number of true positives by the sum of true positives and false positives.

- **Example**
- Suppose you have a model that makes the following predictions:
- True Positives (TP): 70
- False Positives (FP): 30

$$\text{Precision} = \frac{70}{70 + 30} = \frac{70}{100} = 0.7$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Recall

Step 1: The Original Text

"The cat sat on the mat. It was a lazy afternoon. The dog barked loudly. A bird chirped from the tree."

Step 2: Reference Annotations (Actual Boundaries - Ground Truth)

First, we manually identify where the true sentence boundaries are. These are our "reference" or "gold standard" annotations.

- "The cat sat on the mat**|**"
- "It was a lazy afternoon**|**"
- "The dog barked loudly**|**"
- "A bird chirped from the tree**|**"

In this reference, there are **4 actual sentence boundaries**.

- (Boundary 1: after "mat")
- (Boundary 2: after "afternoon")
- (Boundary 3: after "loudly")
- (Boundary 4: after "tree")

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Recall

Step 3: Automatically Estimated Boundaries (System Output)

Now, let's imagine a hypothetical system processes the text and identifies sentence boundaries.

"The cat sat on the mat**|** It was a lazy afternoon. The dog barked loudly**|** A bird chirped from the tree**|**"

The system identified boundaries at these positions:

- After "mat" (Matches a reference boundary!)
- After "loudly" (Matches a reference boundary!)
- After "tree" (Matches a reference boundary!)

Step 4: Count the Components

1.Number of Correctly Identified Sentence Boundaries (True Positives):

1. The system correctly identified the boundary after "mat".
2. The system correctly identified the boundary after "loudly".
3. The system correctly identified the boundary after "tree".

4. Count = 3

2.Total Number of Actual Sentence Boundaries (from Reference):

1. We counted this in Step 2.

2. Count = 4

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Recall

$$\text{Recall} = \frac{\text{Number of Correctly Identified Sentence Boundaries}}{\text{Total Number of Actual Sentence Boundaries}}$$

$$\text{Recall} = \frac{3}{4} = 0.75 \text{ or } 75\%$$

A Recall of 75% means that our system was able to find 75% of the *true* sentence boundaries present in the text.

In this case, it missed one actual sentence boundary (the one after "afternoon").

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

The **F1 score** is a way to combine two important metrics—**precision** and **recall**—into a single number that balances both.

- Precision**: Of all the boundaries your system guessed, how many were actually correct?
- Recall**: Of all the actual boundaries, how many did your system find?

The F1 score is **the harmonic mean** of precision and recall, and is especially useful if you want a single score that reflects both metrics, particularly when there's a tradeoff between them.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

F1 Score in Sentence Segmentation

"The cat sat on the mat. It was a lazy afternoon. The dog barked loudly. A bird chirped from the tree."

1. Reference (Ground Truth) Sentence Boundaries:

After "mat": Yes

After "afternoon": Yes

After "loudly": Yes

After "tree": Yes

2. System Output (Detected Boundaries)

Let's say the system predicts boundaries at:

- After "mat" (**Correct**)

- After "loudly" (**Correct**)

- After "tree" (**Correct**)

- After "afternoon" (**Correct**)

- After "cat" (**Wrong—False Positive**)

- After "barked" (**Wrong—False Positive**)

So, the system finds **4 correct boundaries** and makes **2 extra (incorrect) boundaries**.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

3. Calculate Precision and Recall

True Positives (TP):

4 (correct boundaries found: "mat", "afternoon", "loudly", "tree")

False Positives (FP):

2 (incorrect boundaries: after "cat" and after "barked")

False Negatives (FN):

0 (all actual boundaries were detected; none missed)

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{4}{4 + 2} = \frac{4}{6} \approx 0.666 \text{ or } 66.6\%$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{4}{4 + 0} = \frac{4}{4} = 1.0 \text{ or } 100\%$$

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

4. Calculate F1 Score

$$F1 = 2 \times \frac{0.666 \times 1.0}{0.666 + 1.0}$$

$$F1 = 2 \times \frac{0.666}{1.666}$$

$$F1 = 2 \times 0.3996 \approx 0.799 \text{ or } 79.9\%$$

Metric	Value
Precision	66.6%
Recall	100%
F1 Score	79.9%

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

F1 Score

High Recall (100%): The system found all the true boundaries—none were missed.

Moderate Precision (66.6%): About two-thirds of its predicted boundaries were actually correct. The rest were mistakes.

F1 Score (79.9%): This single number summarizes the system's balance between precision and recall.

NOTE:

F1 will only be high if both precision and recall are high. If either is low, the F1 drops, reflecting the tradeoff!

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Features:

- ❖ In Natural Language Processing (NLP), features are essential components used to represent text and speech data for various tasks, including classification, recognition, and generation.
- ❖ Many features can be applied to both text and speech, as both modalities share certain common characteristics. Here are some features that are commonly used for both text and speech in NLP:

Bag of Words (BoW):

- ❖ BoW represents text by creating a vocabulary of unique words in the corpus and counting their occurrences in each document.
- ❖ For speech, BoW can be applied by transcribing spoken language into text and then using text-based BoW features.
- ❖ **Example:** The sentence "I love cats and dogs" becomes a vector {"I": 1, "love": 1, "cats": 1, "and": 1, "dogs": 1}.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Term Frequency-Inverse Document Frequency (TF-IDF):

- ❖ TF-IDF represents the importance of a term in a document relative to a corpus.
- ❖ It can be used for both text and transcribed speech data to capture the importance of words or terms.
- ❖ Reduces the influence of common words (e.g., "the", "is").
- ❖ **Example:** A word occurring frequently in one document but not others will have a high TF-IDF score.

Word Embeddings:

- Word embeddings like Word2Vec, and FastText represent words in a continuous vector space, capturing semantic relationships.
- Word embeddings can be used for both text and transcribed speech data to capture word-level semantics.
- Word embeddings place similar words like "king" and "queen" close to each other in vector space.

n-grams:

- n-grams represent sequences of n contiguous words or characters.
- They are commonly used in text for capturing local context and can also be applied to transcribed speech data.
- Sequences of n contiguous words or characters. Bigram (2-grams), Trigram (3-grams), etc.
- Captures word patterns and context for better performance in tasks like text classification.
- The sentence "I love cats" can have bigrams like ["I love", "love cats"].

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Part-of-Speech (POS) Tags:

- POS tags represent the grammatical category of each word in a sentence.
- They are used in both text and speech processing to analyze syntactic properties.
- In "The cat runs", cat is tagged as a noun (NN), and runs as a verb (VB).

Phonetic Features:

- For speech data, phonetic features represent acoustic characteristics of speech sounds, such as pitch, duration, and formants (**Formants are frequency peaks in the spectrum which have a high degree of energy. They are especially prominent in vowels. Each formant corresponds to a resonance in the vocal tract**).
- These features can be used for tasks like speech recognition and emotion detection.

Spectrogram Features:

- Spectrogram features represent the time-frequency distribution of audio signals.
- They are used for both speech and environmental sound analysis.

Sentiment Lexicons:

- Sentiment lexicons provide sentiment scores for words.
- They are used in both text and speech sentiment analysis tasks.

Topic Models:

- Topic modeling techniques like Latent Dirichlet Allocation (LDA) can be applied to both text and transcribed speech to identify latent topics in the data.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Syntactic Dependency Features:

Syntactic dependency features represent grammatical relationships between words in a sentence. They are used in both text parsing and spoken language understanding.

Emotion Features:

Emotion features capture emotional content in both text and speech data, helping with tasks like emotion detection or sentiment analysis.

Text-to-Speech (TTS) Features:

TTS features represent acoustic properties of synthesized speech. They are used in TTS systems for generating natural-sounding speech.

Prosodic Features:

Prosodic features capture suprasegmental aspects of speech, such as intonation, rhythm, and stress patterns. They are used in various speech processing tasks, including emotion recognition and speech synthesis.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Processing Stages:

- ❖ Processing stages in Natural Language Processing (NLP) refer to the steps or phases involved in the analysis and understanding of natural language text or speech data.
- ❖ These stages are typically sequential and aim to transform raw language input into structured information that can be used for various NLP tasks.
- ❖ These processing stages form the foundation of many NLP applications, including text classification, machine translation, sentiment analysis, question answering, and more. The choice of stages and techniques depends on the specific NLP task and goals.

Text Acquisition:

The process begins with acquiring raw text or speech data from various sources, such as documents, websites, audio recordings, or chat logs.

Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Text Preprocessing: In this stage, the raw data is cleaned and standardized. Common preprocessing steps include:

- ❖ Tokenization: Splitting text into words or subword units (e.g., by spaces or punctuation).
- ❖ Lowercasing: Converting all text to lowercase for uniformity.
- ❖ Stop Word Removal: Eliminating common words (e.g., "and," "the") that do not carry significant meaning.
- ❖ Stemming or Lemmatization: Reducing words to their base or root form (e.g., "running" to "run").
- ❖ Spell Correction: Fixing typographical errors and misspellings.
- ❖ Noise Removal: Eliminating irrelevant characters or symbols.

Text Representation: In this stage, text or speech data is transformed into numerical or symbolic representations that machine learning models can work with. Common representations include:

- ❖ Bag of Words (BoW): Representing text as a matrix of word frequencies.
- ❖ Term Frequency-Inverse Document Frequency (TF-IDF): Assigning weights to words based on their importance in documents.
- ❖ Word Embeddings: Representing words as continuous vectors in a vector space.
- ❖ Phonemes: Representing speech as sequences of phonemes, the smallest sound units in a language.

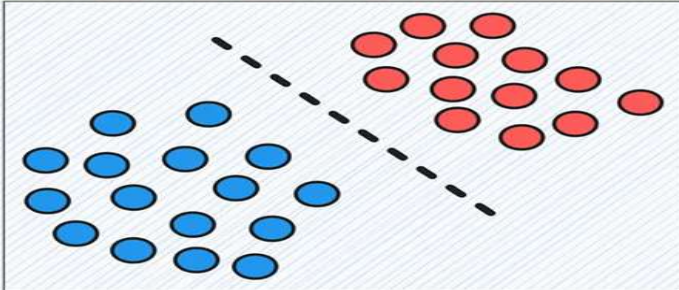
Finding the Structure of Words and Documents



Learning from Experience, Sharing with Passion

Aspect	Generative Models	Discriminative Models
Purpose	Model data distribution	Model conditional probability of labels given data
Use Cases	Data generation, denoising, unsupervised learning	Classification, supervised learning tasks
Common Examples	Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs)	Logistic Regression, Support Vector Machines, Deep Neural Networks
Training Focus	Maximize likelihood of observed data, Capture data structure	Learn decision boundary, Differentiate between classes
Example Task	Image generation, Inpainting (e.g., GANs, VAEs)	Text classification, Object detection (e.g., Deep Neural Networks)

Discriminative Models



Learns the decision boundary between classes

Maximizes the conditional probability: $P(Y|X)$

Directly estimates $P(Y|X)$

Cannot generate new data

Specifically meant for classification tasks

Discriminative models don't possess generative properties

Logistic Regression

Random Forests

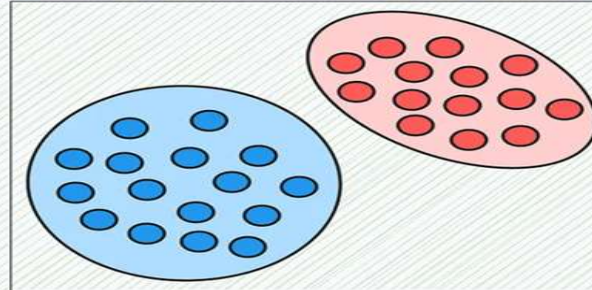
SVMs

Neural Networks

Decision Tree

kNN

Generative Models



Learns the input distribution

Maximizes the joint probability: $P(X, Y)$

Estimates $P(X|Y)$ to find $P(Y|X)$ using Bayes' rule

Can generate new data

Typically, they are NOT used to solve classification tasks

Generative models possess discriminative properties

Hidden Markov Models

Naive Bayes

Gaussian Mixture Models

Gaussian Discriminant Analysis

LDA

Bayesian Networks

